Imperial College London

MEng Individual Project

Imperial College London

Department of Computing

Trajectory-Level Refactoring Analysis and Actionable Explanations Using IDE Traces

Author:
Ethan Hosier

Supervisor:
Dr Robert Chatley

Second Marker:
-

May 23, 2026

Abstract

Your abstract goes here

Acknowledgements

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background

Refactoring is a routine activity in software development: developers restructure code to improve readability, maintainability, and evolvability while keeping externally observable behaviour the same. Traditional definitions emphasise this preservation of behaviour and describe refactoring as a sequence of small, semantics-preserving transformations [1, 2, 3], and subsequent surveys categorise these refactoring operations and the tools that support them [4]. In practice, refactoring is widely encouraged because poorly structured code becomes an obstacle when enhancing and maintaining code, and long-lived systems need continual adjustment (as argued in work building on Lehman's laws and empirical studies of software maintenance). At the same time, refactoring can be costly: it consumes developer time, introduces risk, and is frequently intertwined with other work such as feature development and bug fixing rather than occurring as isolated "refactoring-only" tasks [5, 6].

1.2 Motivation

Most research and tooling that evaluates refactoring focuses on *outcomes* rather than the *process*. A common approach is to measure refactoring success as a change in static quality indicators between a "before" and "after" state: reductions in code smells, improvements in cohesion/coupling, decreases in complexity, or related metrics to do with maintainability. Typical strands of work include metric-driven detection or prioritisation of candidate refactorings (e.g., using metric changes to separate refactoring-like edits from functional ones, or to choose between different refactoring options), as well as refactoring recommendation systems that rank alternatives by their impact on design metrics. More recent work expands the signals used for guidance by incorporating traceability alongside code metrics when ranking refactoring solutions, and exploring how product/process metrics relate to developers' motivations to refactor (e.g., studies building on mined refactorings and repository history, and proposals that use LLMs to determine developers' motivations from commit artefacts). Alongside this, there is a substantial line of work that has improved the ability to *detect* refactorings from code changes, notably by using AST-based differencing and structured matching to infer refactoring operations between revisions (e.g., RefactoringMiner and related approaches), which has enabled large-scale empirical studies of how often refactoring happens, what kinds occur, and in what contexts [7, 8].

However, these perspectives evaluate refactoring as a mapping from an initial program state to a final program state, leaving the *trajectory* between those states largely outside the picture. This matters because developers rarely refactor in a single atomic operation: they work through intermediate states, sometimes take detours, redo work, and often accept temporary degradation (e.g., in readability, modular structure, or test/build stability) before arriving at a final outcome. Two developers can arrive at the same end state via very different paths: one might take lots of disruptive steps, bounce between designs, or create avoidable churn, while another might get there via a steadier, more stable sequence. Existing endpoint-only evaluation cannot distinguish these different cases. Meanwhile, research on process realism suggests that refactoring is often "flossed" into ongoing development instead of being a separate dedicated task, which increases the chance of mixed-intent steps and noisy traces [5]. This makes the quality of the process (how a

developer navigates intermediate states) potentially important for both developer productivity and the health of the codebase as it evolves.

Concurrent work has begun to analyse refactoring trajectories under a state/action/observation formalism, qualitatively classifying failure modes of language-model agents on multi-file refactoring tasks [9] and characterising recurring action patterns in software-engineering agent traces [10]. A recent systematic literature review of LLM-based refactoring research likewise identifies process-aware evaluation as an open gap in the field [11]. As far as we are aware, however, no prior system both (i) scores a developer's refactoring trace against an explicit process-quality metric and (ii) produces counterfactual reference trajectories tied to measurable divergence points. The closest work either synthesises a path to reach an end state (without judging the developer's path) or recommends refactorings on an architectural target (without reference to an observed process). We instantiate the framing in a tool, evaluate it on a 45-session corpus of recorded refactoring sessions with hand-labelled ground truth, and report per-kind precision and recall alongside a sensitivity and ablation analysis of the underlying score formula.

1.3 Research Objectives

This project investigates the following thesis question: **given a known start state and a known end state, can we evaluate the quality of the developer's refactoring process, and identify points where a better process was available?** The core idea is to explicitly model refactoring as a *process* over states, rather than judging it only by comparing endpoints. To do this, the project adopts a state/action/trace framing: snapshots of the codebase are treated as *states*, and developer edits—including IDE refactorings and manual transformations—are treated as *actions* that produce a trace from the start state to the end state.

A core component will be a **process-quality metric** that scores traces according to criteria that will be defined and justified using existing research on code quality metrics, smell detection validity, and process-level signals. The metric is not assumed to be purely structural: it may incorporate multiple terms capturing endpoint improvement, intermediate-state degradation, churn/disruption, and "safety" proxies such as preserving build and test behaviour, reflecting the known limitations of relying on any one signal alone.

To go beyond scoring a single observed refactoring trace, the project will construct *counterfactual* alternatives in the form of other **reference trajectories** between the same start and end states which have been optimised to achieve a higher score under a chosen process-quality metric. This provides a reasonable basis for comparison: analysis can identify *divergence points* where the developer's trajectory deviates from a higher-scoring path, and quantify how much of the gap in resulting refactoring quality is due to each divergence. This extends previous work that synthesises or recommends refactoring sequences: ReSynth synthesises a path to reach a target end state from a developer's partial edits [12], Refactoring Navigator suggests refactoring steps to guide a system toward a desired target at an architectural level [13], and search-based refactoring treats refactoring sequences as an optimisation problem under explicit quality functions [14, 15]. While these systems show that refactoring sequences can be generated and optimised, they do not aim to assess the quality of a developer's chosen process under an explicit process-quality objective. In this project, counterfactual generation supports *process evaluation*: it is used to construct a higher-scoring reference trajectory under the chosen metric and to explain key divergence points, rather than to automate the developer's work or to find any arbitrary sequence of steps that can be followed to reach the end state.

1.4 Contributions

Concretely, this thesis makes the following contributions:

- A process-quality metric for refactoring trajectories, with each term grounded in cited prior work on code quality and developer process signals, and validated by a single-knob sensitivity sweep and a full power-set ablation analysis.
- A first-class divergence-point detector that classifies divergence into four kinds (ORDERING, MANUAL_REFACTOR, REWORK, HYGIENE) and produces a counterfactual reference trajectory for each.

- A 45-session hand-recorded corpus on a purpose-built Java fixture, with multi-label ground-truth annotations, together with the experimental drivers (sensitivity, ablation, divergence) used to evaluate the detector against it.
- An empirical evaluation of the detector that establishes per-kind precision and recall, surfaces what the score formula measurably captures, and identifies which categories of process improvement the formula is structurally insensitive to.

1.5 Approach and Scope

The approach proposed in this thesis targets refactoring as it actually occurs in real-world development, including sequences that involve both IDE-supported refactorings and manually performed edits. It initially focuses on refactorings in a single language (Java) to allow the use of mature parsing infrastructure, established quality metrics, and state-of-the-art refactoring mining (e.g., RefactoringMiner). A refactoring process is represented as a sequence of steps between code snapshots. Steps may correspond to commits or developer-defined checkpoints, and this representation can be refined using higher granularity IDE data where available.

The final system comprises (i) an IDE plugin for capturing refactoring traces and presenting feedback, and (ii) an analysis component that scores the observed refactoring process and finds higher-scoring alternative trajectories between the same start and end states under a chosen process-quality metric. Rather than attempting all possible refactoring operators in a given state, or calculating globally optimal refactoring trajectories, the goal is to produce a plausible reference trajectory which scores higher on the chosen metric, and to explain the points at which the developer's trajectory diverged from it (e.g., where making a different earlier decision could have avoided churn, reduced temporary quality degradation, or preserved build/test stability).

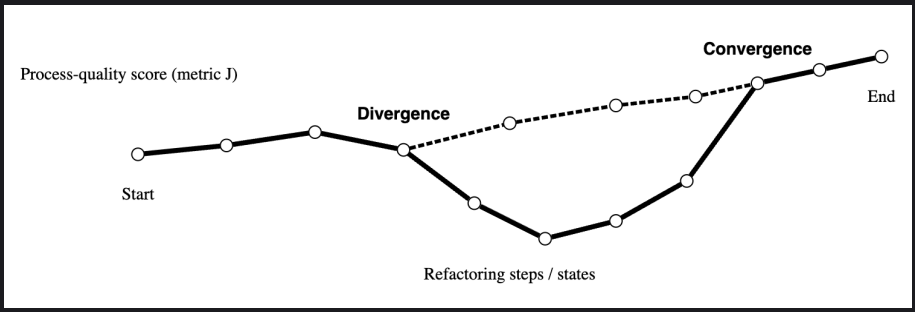

Figure 1.1: Illustration of refactoring trajectories scored by process-quality metric J. The solid line shows the developer's observed trajectory, and the dashed line shows a higher-scoring reference trajectory that diverges at an intermediate state and converges near the end.

1.6 Thesis Roadmap

The remainder of this thesis is structured as follows. Chapter 2 surveys related work in refactoring evaluation, mining, recommendation and synthesis, and positions the present work against that literature. Chapter 3 defines the process-quality metric and the divergence-point detector. Chapter 4 describes the tool's architecture and implementation. Chapter 5 reports experimental results on the 45-session corpus and analyses threats to validity. Chapter 6 concludes and identifies directions for future work.

Chapter 2

Background and Related Work

This chapter reviews the background needed to evaluate *refactoring as a process* rather than just refactoring outcomes. It starts with established definitions of refactoring and the concept of refactoring sequences (§2.1). It then covers the more common outcome-based approach for evaluating refactoring (§2.2), before discussing the quality signals typically used to define "better code", including code smells and static metrics (§2.3). It then looks at methods for detecting and mining refactorings from code changes (§2.4), which is what makes it possible to extract actions and traces from real development histories. The chapter goes on to survey refactoring recommendation and sequence synthesis (§2.5), prior work on capturing developer process traces in the IDE (§2.6), and concludes with the positioning of the present thesis against this body of work (§2.7).

2.1 Definitions and framing

2.1.1 Refactoring as behaviour-preserving transformation

Refactoring is defined as a change to a program's internal structure intended to improve design qualities (e.g. readability, maintainability) while preserving externally observable behaviour. Most definitions emphasise that refactorings are applied as small, semantics-preserving steps and that safety is ensured through maintaining preconditions and/or validation (e.g. testing) [1, 2, 3]. Opdyke's foundational work explains refactorings as program transformations equipped with applicability conditions (preconditions) intended to preserve behaviour under a static approximation [1]. Fowler's catalogue helped popularise refactoring as an incremental activity performed by composing many small operations, grounded in developer-facing mechanics and motivations [2, 3]. Mens and Tourwé later survey the research landscape, bringing together definitions, taxonomies, and tool support, and emphasising the role of precondition checking and program analysis in automated refactoring [4].

In practice, the intent of maintaining behaviour during refactoring is often complicated by the realities of real-world development: refactorings are interleaved with feature and bug-fix work, may be partially performed, and may be mixed with behaviour-changing edits. Consequently, whether a change is "pure refactoring" is often ambiguous when viewed only through version history. This motivates treating behaviour preservation as an *intended property* of refactoring steps, while acknowledging that empirical traces may contain mixed-intent transitions.

2.1.2 Atomic refactorings, smells, and sequences

Research on refactoring often defines a set of *operators* (also described as atomic refactorings) such as Extract Method, Move Method, or Rename that can be composed together into larger transformations [2, 4]. These operators provide (i) a definitive set of actions for tools and (ii) a way to reason about refactoring sequences. The notion of *code smells* is also closely related: these are heuristic indicators that suggest potential design problems and thus potential refactoring opportunities [2, 3] in codebases. While smells are not formal criteria for correctness, they provide a widely-used bridge between developer intuition and measurable signals (discussed further in §2.3).

A key distinction of this thesis is between evaluating refactoring as an *endpoint mapping* (before vs after) and evaluating refactoring as a *trajectory* (a sequence of intermediate states). Even if

two trajectories share the same start and end states, they may differ substantially in intermediate degradation, churn, or disruption, which can be consequential for productivity and risk during the refactoring process.

2.1.3 State–action–trace formalism

To explicitly represent refactoring as a process, this thesis adopts a state/action/trace framing aligned with the view of refactoring as sequential operator application [2, 4]. Let a codebase snapshot be a *state* $s \in \mathcal{S}$ (e.g. the repository at a particular point in time). Let an *action* $a \in \mathcal{A}$ denote a developer-performed edit step, which may be an IDE-supported refactoring operation, a manually performed transformation, or a mix of the two. A refactoring process is then a *trace*

$$\tau = (s_0, a_0, s_1, a_1, \ldots, a_{T-1}, s_T),$$

where each a_t transforms state s_t into s_{t+1}. Under this framing, "process quality" can be treated as a property of τ (and potentially its prefixes), rather than a property only of (s_0, s_T).

Two practical issues arise from this. First, the granularity of steps (what constitutes one a_t) is not constant: steps may be commits, IDE events, or developer-defined checkpoints. Second, action labels may be partially observed or inferred (e.g. via refactoring detection tools). These motivate our review of refactoring mining (§2.4) and shape how we design process-quality metrics (§2.3).

This state/action/trace framing is consistent with recent software-engineering work that models agent and developer behaviour as a partially-observable Markov decision process, in which a trajectory is a sequence of (action, observation) pairs [9, 10]. We retain the explicit s_t notation in addition to actions because intermediate states-rather than only observations-are the primary object scored by the process-quality metric defined in the methodology chapter.

2.2 Outcome-based refactoring evaluation

2.2.1 Endpoint-centric quality improvement

A common way to evaluate refactoring is by comparing quality indicators computed on a "before" snapshot and an "after" snapshot. In this framing, refactoring is deemed successful if it improves a chosen set of quality proxies (e.g. reduced coupling, increased cohesion, reduced complexity, fewer code smells). This endpoint-focused perspective appears in both empirical studies of refactoring impact and in automated recommendation systems that rank refactoring candidates by their predicted improvement.

A representative example is *search-based refactoring*, which frames refactoring as an optimisation problem over sequences or sets of refactoring operations. Harman and Tratt demonstrate multi-objective search at the design level, explicitly treating coupling and cohesion as competing objectives and producing Pareto-optimal trade-offs [14]. Later work extends this to many-objective settings and introduces constraints or preferences that reflect practical concerns such as test coverage or prioritisation [15]. These approaches make it explicit that "better" is inherently multi-dimensional, and collapsing everything into a single quality score can be overly restrictive.

This same outcome-based ranking also appears in recommender systems that evaluate alternative refactoring solutions. For example, some systems extend beyond structure-based metrics by incorporating additional information such as requirements traceability, arguing that designs can be "better" not only in properties derivable from the code structure, but also in maintainability tasks like impact analysis [16]. More recently, researchers have revisited the problem of selecting among large sets of refactoring candidates and investigated which code characteristics tend to correlate with positive or negative impacts in practice [17].

2.2.2 What endpoint evaluation captures-and what it misses

Endpoint-based evaluation is best-suited to answer questions of the form: "is the end state better than the start state under metric Q?" It provides a simple basis for automated recommendation (simulate each candidate and then choose the one that maximises ΔQ) and for empirical analysis (associate refactoring events with changes in quality proxies). It also aligns with the fact that many quality metrics are defined as functions of a single codebase snapshot.

The downside is that it ignores the refactoring process itself, and therefore cannot distinguish between trajectories that reach the same endpoint. In real refactoring, intermediate states can temporarily degrade in ways that matter: code may become harder to understand, modular structure may be disrupted, or build/test stability may regress before it improves again. Furthermore, the *cost* of reaching an improved endpoint is not solely captured by the endpoint metrics alone: disruptive changes can increase churn, complicate review, and introduce integration risk even if final static metrics improve. Empirical work on modularity improvement highlights this trade-off between optimisation and disruption, showing that metric gains can come with substantial structural disturbance [18]. This motivates evaluating not only final quality, but also the *trajectory cost* and stability of the path taken.

These limitations are central to the thesis: the goal is not to replace outcome-based quality metrics, but to use them within a process-level evaluation that can treat intermediate degradation, rework, churn, and stability as first-class terms in a process quality objective.

2.3 Code smell detection and quality metrics

2.3.1 Smell detection as operationalising design issues

Code smells are a common way to link between qualitative design concerns and measurable indicators [2]. Smell detectors typically detect these smells via rules or learned models. They usually rely on structural properties (e.g. size, coupling, cohesion) and, increasingly, lexical and semantic features also. A prominent rule-based system is DECOR, which defines smells and design antipatterns using a set of measurable symptoms and corresponding detection rules [19]. DECOR is widely used in empirical studies because its definitions are explicit and it can be executed efficiently at scale.

Despite being useful, smell detectors are not ground truth. Comparative studies often show disagreement between tools in both precision and recall, and that performance is highly variable across different systems and smell types [20]. These differences are largely derived from how each tool defines smells in practice: thresholds, feature sets, and implementation details. Furthermore, detectors may produce false positives: code sections flagged as "smelly" when there is no clear negative impact or that reflect acceptable design trade-offs [21]. For process evaluation, these issues imply that smell counts should be treated as one signal among several rather than as a sole objective: combined with complementary indicators (structural metrics, readability proxies, and process-level safety signals), they contribute useful information without carrying the result alone. The process-quality metric defined in the methodology chapter adopts this combinatorial approach, treating smell-based terms as one component of a multi-term score rather than the dominant signal.

2.3.2 Quality metrics: structural and readability-oriented signals

Static metrics offer another way to quantify design quality at the granularity of classes, methods, and packages. Classic metric families quantify coupling, cohesion, complexity, and size, and are widely used as proxies for maintainability. Recent work has also argued for using readability-oriented signals that capture aspects not well-modelled by purely structural measures. For example, readability models that combine structural features (e.g. line structure) with textual features (such as identifier patterns) tend to align better with human judgements of readability [22]. These signals are particularly relevant for evaluating refactoring because readability and understandability are common motivations for refactoring and are often affected by intermediate states.

2.3.3 Implications for process-quality metric construction

The process-quality metric $J(\tau)$ used in this thesis will draw on these signals, but it must also account for their limitations. The literature points to three key design principles:

1. **Use multiple signals.** No single metric or smell detector provides reliable ground truth across contexts [20, 21]. Combining complementary signals can reduce sensitivity to tool-specific noise.

2. **Differentiate endpoint improvement from trajectory cost.** Outcome-based improvements remain valuable, but disruption and intermediate degradation can be significant even when the final snapshot improves [18].

3. **Prefer interpretable components.** For a process critique to be actionable, the metric should be decomposable into terms that can be explained (e.g., readability dip, churn spike, smell increase), rather than a single opaque score.

A further practical implication is that process evaluation should include an explicit notion of developer **effort**, such as the length of the trajectory (number of steps) and/or the magnitude of edits required between successive states. While fewer steps are not inherently better (e.g., small "micro-steps" may reduce risk), step count and edit effort provide a natural regulariser against unnecessary detours and rework, complementing quality-improvement terms and measures of disruption. Cost terms like these are commonly treated as optimisation objectives alongside structural quality goals in search-based refactoring and recommendation settings (e.g., minimising change while improving quality), and are therefore suitable candidates for trajectory-level scoring [14, 15].

Table 2.1 summarises candidate signals, their typical benefits and limitations, and examples of available tooling in the Java ecosystem. The purpose of the table is not to fully specify the final form of the process-quality metric $J(\tau)$ (e.g., precise term weights and normalisation), but to motivate the choice of component signals and the design principles that the final metric will comprise. A precise definition of $J(\tau)$ is given in the methodology chapter and used consistently throughout the thesis evaluation.

Signal	**Pros**	**Cons / threats**	**Typical tooling**
Structural metrics (coupling/cohesion/ complexity/size)	Widely used; efficient; interpretable at class/method granularity	Proxy validity depends on context; may incentivise disruptive restructuring; metric interactions	Static analysis metric extractors; IDE/CI integrations
Smell counts / severity (rule-based)	Directly targets design issues; aligns with developer's motivations	Detector disagreement and false positives; threshold sensitivity; context dependence	DECOR-style detectors [19]; other smell tools [20]
Readability models	Closer to human comprehension than structural-only metrics; captures lexical/style clues	Model generalisation and dataset bias; may be language/style dependent	Readability predictors [22]
Churn / disruption (LOC changed, file moves/renames)	Captures cost and instability of change; naturally trajectory-oriented	Churn may be necessary for large refactors; can penalise required restructuring	VCS mining; diff statistics
Process length / effort (number of steps; edit magnitude)	Trajectory-native cost signal; discourages detours/rework; easy to compute	Sensitive to step granularity (commits vs. checkpoints vs. IDE events); may penalise safe micro-steps; requires calibration	VCS checkpoints; IDE logs; diff/AST edit size
Build/test preservation proxies	Direct safety signal; aligns with "behaviour-preserving intent"	Requires runnable builds/tests; noisy due to flaky tests; environment dependence	CI logs; local build/test runs
Traceability-augmented signals	Reflects maintenance tasks beyond code structure; complements metric-only ranking	Requires trace links (often unavailable); domain-specific assumptions	Traceability-based ranking methods [16]

Table 2.1: Candidate signals for constructing a process-quality objective. The thesis combines endpoint quality change with trajectory-oriented cost and stability terms, rather than relying on any single indicator.

2.4 Refactoring detection and mining from code changes

2.4.1 From diffs to refactoring actions

To evaluate a process trace in practice, one usually needs to either (i) capture developer actions directly (for example, IDE event logs) or (ii) infer actions from the differences between snapshots (like commits). Much of this work focuses on the second case: given two versions of a program, infer whether a refactoring happened and, if it did, identify which refactoring types occurred.

A key technical enabler is *structured code differencing*, usually built on abstract syntax trees (ASTs). This makes it possible to detect moves, renames, and other non-local edits that line-based diffs tend to miss. GumTree is a representative AST differencing approach: it builds fine-grained mappings between AST nodes and produces edit scripts that line up better with the structural changes developers actually intended [8]. This kind of differencing is often used as a base layer for refactoring detection.

Building on these foundations, modern refactoring miners detect refactoring instances by combining structural mappings with refactoring-specific heuristics. RefactoringMiner is a well-known tool that detects a wide range of refactoring types by analysing successive revisions, using structured matching to relate program entities across versions [7]. RefDiff takes a similar direction: it detects common refactoring types in version histories using similarity heuristics and static analysis [23]. Together, these tools support large-scale mining studies and can also provide an *action vocabulary* for traces, for example by labelling a transition $s_t \to s_{t+1}$ with a multiset of detected refactoring operations.

2.4.2 Representation, coverage, and typical failure modes

Refactoring detection tools typically output refactoring instances using a taxonomy of operator types (e.g., Move Method, Extract Class) and the program entities involved. This is useful because it matches the operator-based view of refactoring used in catalogues and surveys [2, 4]. However, the literature repeatedly reports several limitations, and these are especially important when doing process evaluation.

Mixed and tangled changes. Version control commits often mix refactoring and functional edits, or they bundle multiple unrelated activities together. In these cases, a commit-level transition can include both behaviour-preserving and behaviour-changing edits, and refactoring detection may only return partial results or it may be ambiguous. This reduces how interpretable the actions extracted from snapshots are, and it motivates careful trace construction (for example, using finer checkpoints) and methods that are robust to noisy labels.

Incomplete coverage and composition effects. Detectors only cover a finite set of operators, and refactorings can also be composed (e.g., move+rename) in ways that make matching harder. Some refactorings are done manually without tool support, and they might not fit the patterns that detectors expect. For process evaluation, this means that "actions" should be treated as only partially observed: detected refactorings are useful as informative features, but they are not an exhaustive description of developer intent.

Behaviour preservation is not verified. Detection is usually structural: it infers that a change *looks like* a refactoring, but it does not prove that behaviour was preserved. Because of this, downstream evaluation should not treat "detected refactoring" as the same thing as a "safe refactoring". It should also use independent safety proxies where possible (for example, whether the build passes or tests still pass).

2.4.3 Tie-in to this thesis

Refactoring mining is important infrastructure for building traces and for characterising actions at scale [7, 23]. However, the contribution of this thesis is not to improve refactoring detection accuracy. Instead, mined refactorings are used as part of the observational layer: they provide labels and features that describe transitions between states, which can support constructing a process-quality metric and generating reference trajectories. The next part of this background builds

on these ideas to discuss prior work on refactoring sequence generation and navigation, which is directly relevant to constructing higher-scoring reference trajectories under a chosen objective.

2.5 Refactoring recommendation and search/synthesis

The previous sections motivate two requirements for process-level refactoring evaluation: (i) a way to represent and score a developer trace, and (ii) a way to construct counterfactual *alternative* trajectories for comparison. A substantial body of work already generates refactoring suggestions and sequences, but it usually targets objectives that are different from evaluating a developer's process. Broadly, existing approaches fall into three overlapping families: (a) *synthesis* approaches that reconstruct or complete refactoring sequences from examples or partial edits; (b) *search-based* approaches that optimise refactoring sequences under quality objectives; and (c) *interactive* approaches that adapt recommendations based on developer feedback. This section reviews these families and explains how they can support, but do not directly solve, process-quality evaluation.

2.5.1 Synthesis and path completion from partial edits

One way to generate counterfactual alternatives is to treat refactoring as a program transformation problem: given a start program and a target (or partially edited) program, synthesise a sequence of refactoring operators that carries out the transformation. ReSynth is a representative system in this space [12]: it synthesises a sequence of IDE-supported refactorings consistent with a developer's partial edits, constraining the search by the entities involved in those edits and by the operators' preconditions. Its objective is *reachability and consistency* (does some operator sequence reproduce the developer's edits?) rather than process-quality evaluation. The general lesson for this thesis is that refactoring can be framed as a structured search problem over refactoring operators, and that the search space can be constrained using evidence from the developer trace itself rather than enumerated naively.

A closely related line of work generalises "from examples" synthesis beyond classical refactorings. REFAZER learns program transformations from input-output examples of code edits [24]. Instead of assuming a fixed refactoring catalogue, it synthesises AST-level rewrite rules from a small number of examples and can apply them to automate repetitive edits. While REFAZER is not a refactoring miner in the narrow Fowler sense, it is relevant to counterfactual generation in two ways: (i) it offers a way to infer candidate transformations without predefining an exhaustive operator set; and (ii) it shows that "plausible alternatives" can be derived from edit evidence, which may help bridge the gap between IDE-supported refactorings and manually performed edits.

Relevance and limitations for process evaluation. Synthesis systems mainly aim to *reach* a target program (or a class of targets that are consistent with partial edits) and to do so using sequences that satisfy refactoring preconditions [12]. The objective is reachability and consistency, not evaluating the quality of a developer's chosen trajectory. In particular, synthesis does not completely answer whether the developer's intermediate decisions were efficient, low-churn, or risk-minimising under a chosen process objective. Still, synthesis is useful as a source of *candidate alternative trajectories*: it provides a principled way to generate sequences that are likely to be feasible and realistic in a development environment.

2.5.2 Search-based optimisation of refactoring sequences

A second family treats refactoring recommendation as an optimisation problem. Search-based refactoring methods explore sequences or sets of refactorings and then select those that optimise one or more objective functions based on quality metrics [14]. Harman and Tratt argue explicitly for multi-objective formulations (e.g., coupling vs. cohesion) and produce Pareto fronts of trade-offs, rather than forcing design quality into a single scalar [14]. These methods are appealing because they make "better" more measurable, and they can produce many candidate solutions that show the trade-offs more clearly.

Later work extends these formulations to more realistic settings by adding extra objectives. For example, many-objective approaches include not only quality improvements, but also concerns like prioritisation and how refactoring effort is spread across the codebase [15]. These objectives are close to process evaluation because they start to encode costs and constraints that show up

during development (e.g., avoiding refactoring that is too concentrated, or respecting priorities), instead of treating refactoring as only a structural optimisation.

Search-based techniques also appear in work that looks at the tension between metric improvement and disruption. Empirical studies of modularity optimisation show that maximising cohesion/coupling improvements can cause substantial disruption compared to the original modular structure [18]. This line of work continues into the present: recent many-objective formulations explicitly treat *architectural distance*-a direct disruption proxy-as a first-class Pareto objective alongside structural quality [25], and related work has extended the framework with socio-technical cost proxies such as reviewer availability [26]. This matters for process evaluation: even if an endpoint configuration is "better" under structural metrics, the path to reach it can be very disruptive, and disruption itself may be a cost term that process evaluation should capture.

Relevance and limitations for process evaluation. Search-based refactoring provides two key ingredients: an explicit objective function and a mechanism to generate candidate sequences. However, most work evaluates the *recommended* refactoring sequence, not a human trace. The algorithm's output is treated as the solution, and success is measured by the improvement it produces (or by user study preference for the recommendation). This is different from the present thesis, where the goal is to evaluate a developer's observed trajectory *relative to* alternative trajectories under an objective that is meant to capture process quality. Still, search-based methods give a natural way to construct a *reference trajectory*: given a start state (and potentially an end constraint), they can produce a plausible trajectory that scores highly under the chosen process objective and can act as a baseline for comparison.

2.5.3 Goal-directed navigation and architectural refactoring

A third family focuses on guiding refactoring toward a target design. Refactoring Navigator is one of these: it takes a current system and a desired target architecture/design, then recommends a sequence of refactoring steps using a search-based strategy, presented through a navigation metaphor [13]. The key idea is to guide developers step-by-step toward a goal, rather than just ranking isolated refactorings. Controlled evaluations report large reductions in task time and manual edits compared to unguided refactoring [13], which suggests that sequence-level guidance can change how developers actually refactor.

Goal-directed refactoring is particularly relevant for the "grand scheme" view of this thesis: it is natural to compare an observed developer trajectory to a higher-scoring "reference" trajectory that reaches a comparable end state. Navigation systems show that it is feasible to produce these stepwise trajectories, and to map high-level goals down to low-level refactoring actions.

Relevance and limitations for process evaluation. Navigation systems typically optimise toward reaching a target design, and they may use design metrics to guide progress [13]. They are not built to evaluate a human's process quality, and they do not try to explain where a human trace diverged from a better trajectory. However, they provide strong evidence that (i) refactoring can be treated as a sequential decision problem, and (ii) sequence-level suggestions can be computed and presented in ways that developers can actually use.

2.5.4 Interactive and developer-in-the-loop recommendation

Purely automatic optimisation can end up misaligned with developer intent, constraints, or context. Interactive refactoring recommenders address this by incorporating user feedback during the search process. Alizadeh and Kessentini propose an interactive and dynamic search-based approach that presents refactoring recommendations iteratively and updates the search based on developer decisions [27]. This highlights that refactoring recommendation is not only about finding a high-scoring sequence under fixed metrics; it also involves keeping recommendations aligned with developer preferences and constraints as they evolve during a task.

Relevance and limitations for process evaluation. Interactive recommenders are not evaluation tools, but they are closely aligned with the instrumentation and delivery mechanism envisioned by this thesis. If process evaluation is delivered in an IDE, the feedback has to be understandable, minimally disruptive, and sensitive to developer context. Work on interactive recommendation

provides ideas for presenting sequence-level suggestions and for building feedback loops into the developer's workflow [27]. The remaining open problem is to shift from "recommend to optimise" to "compare and evaluate an observed trajectory relative to alternative, counterfactual trajectories".

2.5.5 Beyond structural metrics: alternative objectives for ranking refactorings

A recurring theme in recommendation research is that structural metrics alone may not be enough to capture what makes a refactoring "good". One example is ranking refactoring candidates using requirements traceability alongside code metrics. Approaches that include traceability treat "quality" as partly determined by how well requirements map to code, and they are motivated by maintenance tasks like impact analysis and comprehension [16]. This supports the idea that process-quality objectives should not be assumed to be purely structural: the relevant costs and benefits of refactoring can include non-structural properties and context-specific constraints.

Recent work also looks at developer motivations for refactoring, and tries to align recommendations with what developers are likely to prioritise. Studies that mine refactoring operations and repository history investigate product/process signals that correlate with refactoring activity and with developer intent, including proposals that use LLMs to infer motivations from commit artefacts [28, 29]. While these works mostly focus on *when* and *why* refactoring happens, rather than *how well* it is carried out as a process, they suggest that developer-aligned objectives may need to incorporate process signals and contextual information.

2.5.6 Synthesis: what sequence-generation work enables, and what remains missing

Sequence generation and recommendation systems establish that it is feasible to:

- generate refactoring sequences that realise a transformation or reach a target (synthesis/navigation) [12, 13],
- optimise sequences under explicit metric objectives (search-based refactoring) [14, 15],
- incorporate developer feedback into iterative suggestion mechanisms (interactive recommendation) [27],
- and broaden "quality" beyond structural metrics where additional artefacts exist (e.g., traceability) [16].

However, these systems do not directly answer the two central questions posed by process-level evaluation:

1. *Was the developer's observed trajectory good under an explicit process-quality criterion?*
2. *Where did the developer's trajectory diverge from a higher-quality trajectory, and can that divergence be explained?*

The distinction is subtle but important. Recommendation and synthesis work typically treats the algorithm's output as the primary object and evaluates its outcome or usefulness. Process evaluation instead treats the *developer trace* as the primary object and seeks to compare it against counterfactual trajectories constructed under a process-quality objective.

2.5.7 Recent trajectory-formalism work in software engineering

Recent work has begun to formalise software-engineering agent behaviour as a trajectory of (action, observation) pairs in a POMDP-shaped environment [9, 10, 30], and a recent systematic literature review of LLM-based refactoring research identifies process-aware evaluation as an open gap in the field [11]. These developments confirm that the trajectory framing is gaining traction in concurrent software-engineering research, but the work to date targets LM-agent behaviour and benchmark task success rather than human developer process quality. Empirical observational studies independently confirm that real-world refactoring proceeds as ordered sequences of operations interleaved with other change activities [31], motivating the trajectory-level framing adopted in this thesis.

2.6 Process-aware work and IDE trace capture

Evaluating a refactoring process requires access to traces: sequences of intermediate states and actions. There are two broad sources of traces: (i) version control histories (commits/checkpoints), and (ii) IDE-level interaction logs that capture finer-grained behaviour. This section focuses on IDE logging and process-aware studies, since they show it is feasible to capture refactoring traces at a useful granularity, and they also highlight practical limitations that end up determining evaluation design.

2.6.1 Logging developer interactions in IDEs

Multiple studies have collected large-scale logs of developer interactions to mine workflows and behavioural patterns. Damevski et al. present an approach for mining sequences of developer interactions in Visual Studio using telemetry-like event streams, with the aim of discovering frequent patterns of usage without necessarily recording the full code content [32]. This is relevant for trace collection because it shows that useful traces can be captured as structured events (e.g., commands, navigation actions, edits) while still maintaining the developer's privacy.

Poncin et al. use Eclipse plugins to collect detailed event logs and then apply process mining techniques to infer developer workflows [33]. This gives a clear example of turning low-level interaction events into higher-level activity models, such as sequences of edits, navigation, compilation, and tool invocations. It also shows some of the practical challenges: event streams are noisy, developers multitask, and mapping raw events to semantically meaningful steps needs careful segmentation and interpretation.

More recent infrastructure work has pivoted toward capturing developer-LLM interactions rather than refactoring-specific developer process traces. *CodeWatcher* is a representative example: a lightweight client-server VS Code extension that captures fine-grained, semantically labelled IDE events (LLM-tool insertions, deletions, copy-paste, focus shifts) with timestamps to support post-hoc reconstruction of coding sessions [34]. While the underlying telemetry mechanisms are directly applicable to refactoring traces, the 2023–2025 literature on IDE-side activity mining for refactoring specifically has been quiet-the broader process-mining-in-SE community has migrated toward repository and CI/CD logs rather than IDE event streams, and recent activity-recognition work targets LLM-coding patterns rather than refactoring sequences. Classical IDE-side activity mining for refactoring trajectories therefore remains an area where canonical references are over seven years old.

2.6.2 Granularity, segmentation, and semantic interpretation

IDE logging raises the question of what counts as a "step" in a refactoring process. At commit granularity, steps are coarse and often tangled with non-refactoring changes. At event granularity, steps are extremely fine and need to be aggregated into coherent actions. The process mining and interaction analysis literature highlights that segmenting activity into tasks/episodes is not trivial [33, 32]. For refactoring in particular, a single conceptual refactoring can involve several micro-actions (navigate, edit, run a refactoring tool, adjust code, run tests). On the other hand, a short burst of edits might correspond to multiple actions.

A process-evaluation system therefore has to choose a step definition that is both (i) practical to extract reliably and (ii) meaningful enough to allow for interpretation.

2.7 Summary and positioning: toward process-quality evaluation

The literature discussed in §2.1–§2.6 establishes a strong foundation for modelling and measuring refactoring, but it also leaves open a clear space for process-level evaluation.

First, refactoring is well-defined as behaviour-preserving and supported by catalogues of operators and by tool-based precondition checking [1, 2, 4]. Second, most evaluation work is still focused primarily on endpoints: it assesses refactoring success by looking at changes in quality indicators between start and end snapshots, including structural metrics and smell-based proxies [14, 19]. These signals are useful, but they are imperfect and can disagree across tools, which motivates combining multiple signals and metrics [20, 21, 22]. Third, refactoring mining and AST

differencing provide practical ways to infer refactoring actions from code changes, which enables trace reconstruction from repository histories [7, 8]. Fourth, recommendation and synthesis systems show that it is feasible to generate refactoring sequences and plan toward final goal states [12, 13, 27], and that "quality" may need to include contextual objectives beyond just structural metrics [16, 28, 29]. Finally, IDE logging research shows that fine-grained traces can be captured and analysed, but it also highlights challenges around the segmentation and interpretation of them [33, 32, 35].

While there has been substantial research on *detecting* refactorings, *recommending* refactorings, and *synthesising* refactoring sequences, as far as we are aware no prior system both (i) scores a developer's refactoring trace against an explicit process-quality metric and (ii) produces counterfactual reference trajectories tied to measurable divergence points. The closest existing work-sequence synthesis, search-based recommendation, and the recent trajectory-formalism studies surveyed above-generates or analyses sequences as the primary object, treating the algorithm's output as the solution rather than evaluating a developer's chosen path against alternatives. Existing research generally does not define and evaluate a criterion for "good refactoring process" that penalises intermediate degradation, disruption, or instability, nor does it provide a specific method to compare a developer's refactoring trajectory against alternative, higher-quality trajectories (relative to the chosen metric) and to explain divergence points from those trajectories.

This thesis addresses this gap by treating refactoring as a trajectory optimisation and comparison problem: given a start state, an end state (or end constraint), and an observed trace, defining a process-quality objective and constructing a higher-scoring reference trajectory for comparison. This leads to the overall thesis question: *given a known start state and end state, can we produce actionable explanations of a developer's refactoring process by constructing a plausible reference trajectory and highlighting where and why the developer's path diverged?*

Chapter 3

Methodology

This chapter defines the three main contributions of this thesis: the process-quality score, the divergence-point detector, and the per-kind alternative trajectory synthesisers. §3.1 sets up the formalism that frames the rest of the chapter. §3.2 and §3.3 define the process score and its weights; §3.4 defines the cleanliness sub-score the process score consumes. §3.5 defines the divergence-point detector, and §3.6–§3.9 define the four synthesisers that produce alternative reference trajectories. §3.10 describes the validator that checks each alternative still reaches the same end state. §3.11 describes the rater study used to validate the labelled sessions in Chapter 5 (§5.2.2).

This chapter focuses on what the algorithms do and why they are correct. How they are made fast in practice - the JDT batch-session reuse, single-worktree borrow, and git-checkout backtracking - is covered in Chapter 4 (§4.6).

3.1 Formal definitions and notation

The formal definitions in this section build on the state/action/trace framing from §2.1. A refactoring trajectory is a sequence

$$\tau = (s_0, a_0, s_1, a_1, \ldots, a_{T-1}, s_T),$$

where each s_t is a code snapshot (a git commit SHA paired with the working-tree contents) and each a_t is a developer action that turns s_t into s_{t+1}. Actions are either IDE-supported refactoring operations or manual edits; in practice a single action can carry both a structured refactoring spec (recovered by RefactoringMiner [7]) and some leftover unstructured edits. We treat this as one action with two parts rather than two separate actions.

The object actually being evaluated is the analysis-report structure produced by the tool. An *analysis report* for a trajectory τ is a tuple $(\tau, \mathcal{M}, \mathcal{D}, \mathcal{A})$, where:

- $\mathcal{M} = \{m_0, m_1, \ldots, m_T\}$ is metrics for a given state. Each m_t contains the cleanliness sub-score $C(s_t)$ together with the raw signals it is built from (PMD violations, CPD duplications, Chidamber–Kemerer measures, build/test outcomes, the action's refactoring spec, and commit and test event metadata).
- $\mathcal{D}$ is the set of divergence points the detector emits for τ (§3.5).
- $\mathcal{A}$ is the set of alternative reference trajectories synthesised at those divergence points (§3.6 onwards). Each $\tau^* \in \mathcal{A}$ has the same tuple shape as the user's report, so alts can be scored under the same machinery.

The process-quality score $J(\tau)$ is computed over the full trajectory using $\mathcal{M}$, and is the focus of §3.2–§3.3. Table 3.1 summarises the notation used throughout the chapter.

3.2 The process score

A note on what is claimed and what is not. The process score is presented here as internally consistent and theoretically grounded against the literature reviewed in Chapter 2. The thesis does

Symbol	Meaning
$\tau = (s_0, a_0, s_1, \ldots, a_{T-1}, s_T)$	Refactoring trajectory
s_t	State (code snapshot) at step t
a_t	Action at step t (IDE refactor or manual edit)
$J(\tau)$	Process-quality score, clamped to $[0, 100]$
$C(s)$	Cleanliness sub-score of state s
$\Delta C(\tau) = C(s_T) - C(s_0)$	Cleanliness gain across trajectory
$\beta = 50$	Baseline anchor in the process score
W_{x}	Process-score weight for term x
$r_b,\ r_{st},\ r_{mi}$	Laplace-smoothed rate terms (broken, skip-tests, manual-when-IDE)
$n_{cg}(\tau)$	Commit-gap event count
$\sigma_l(\tau)$	Step-savings bonus for alternative trajectories
DP	Divergence point
τ^*	Alternative reference trajectory
$\mathcal{K}$	Divergence-kind set = {ORDERING, MANUAL_REFACTOR, REWORK, HYGIENE}

Table 3.1: Notation used throughout the methodology chapter.

not claim that the score is uniquely-correct or calibrated against an external outcome variable; no held-out ground truth for "true refactoring-trajectory quality" exists in this problem space. The empirical evidence presented in Chapter 5 (sensitivity sweep, ablation, inter-rater agreement on the kind classifier) speaks to robustness, term-importance, and the kind classifier's accuracy — not to whether the production weight vector is the correct one to use.

The process score $J(\tau)$ is the main thing this chapter defines. It combines six weighted terms across the trajectory and clamps the result to a $[0, 100]$ range so trajectories of different lengths can be compared on the same scale:

$$J(\tau) = \text{clip}_{[0,100]}\Big(\beta + W_{\text{G}}\,\Delta C(\tau) - W_{\text{B}}\,r_b(\tau) - W_{\text{ST}}\,r_{st}(\tau) - W_{\text{MI}}\,r_{mi}(\tau) - W_{\text{CG}}\,n_{cg}(\tau) + W_{\text{L}}\,\sigma_l(\tau)\Big) \tag{3.1}$$

where:

- $\beta = 50$ is the baseline anchor. A trajectory with neither cleanliness gain nor process damage scores 50.
- $\Delta C(\tau) = C(s_T) - C(s_0)$ is the cleanliness gain from start to end (§3.4).
- $r_b(\tau) = b_{\text{ms}}(\tau)/e_{\text{ms}}(\tau)$ is the time-weighted broken fraction: the proportion of trajectory wall-clock time during which the build or test suite was broken.
- $r_{st}(\tau)$ and $r_{mi}(\tau)$ are Laplace-smoothed event-rate terms of the form $r_\bullet(\tau) = (k+1)/(n+2)$, where k is the number of penalised events and n the number of opportunities. Both default to zero when there have been no events of either polarity yet, so a single bad-out-of-one event does not trigger a full penalty.
- $n_{cg}(\tau)$ is the commit-gap event count: one event per stretch of ≥ 6 green-refactor checkpoints between user commits.
- $\sigma_l(\tau)$ is the alt-only step-savings bonus. It only contributes when τ is a synthesised alternative trajectory that reaches the same end state in fewer steps than the user.

Three design choices shape this formula. First, an *anti-gaming axiom*: the broken-state penalty W_{B} has to outweigh any single-step cleanliness gain $W_{\text{G}} \cdot \Delta C$ a developer could plausibly engineer by trading transient correctness for structural improvement. The severity ordering $W_{\text{B}} : W_{\text{ST}} : W_{\text{CG}} = 28:14:7$ enforces this in the weights of §3.3. Second, the Laplace smoothing keeps the rate terms from misbehaving on very short trajectories where one bad event would otherwise dominate. Third, the $[0, 100]$ clamp makes scores comparable across trajectories of different lengths, but it does introduce saturation: when a user's trajectory is already near 100, any alt's improvement gets compressed against the ceiling. The sensitivity experiment in §5.1.1 tracks this and reports it as a known limitation of the score formula.

3.3 Process-score weights

Table 3.2 lists the six production weights of Equation (3.1) and the literature that backs each one. Their empirical contribution to the formula's ranking output is left to the ablation analysis in §5.1.2, which runs the full power-set of weight subsets ($2^6 = 64$ variants per fixture) and reports both solo and leave-one-out recovery against the production ranking; the literature column below establishes that each weight is grounded in prior work, and the ablation chapter establishes that the weights are doing the work they claim to.

Term	Weight	Cited justification
W_{G} (gain)	50	Cedrim et al. 2017 [36]; Paixão et al. 2017 [18]
W_{B} (broken)	28	Paixão et al. 2017 [18]; Gautam et al. 2025 [9] (FM #2)
W_{ST} (skip-tests)	14	Murphy-Hill et al. 2009 [5]
W_{MI} (manual)	11	Negara et al. 2013 [37]; Vakilian et al. 2012 [38]
W_{L} (length)	11	Harman & Tratt 2007 [14]; Mohan & Greer 2019 [15]
W_{CG} (commit-gap)	7	Tao & Kim 2015 [39]; Di Biase et al. 2019 [40]; Herzig & Zeller 2013 [41]; Kudrjavets et al. 2022 [42]

Table 3.2: Production weights for Equation (3.1) and the literature each weight rests on. Empirical contribution of each term to the formula's ranking output is reported in the ablation analysis of §5.1.2.

The rest of this section walks through each weight. Two of them (W_{MI} and W_{CG}) have a citation gap: the literature supports the idea of the penalty, but does not directly measure exactly what the weight captures. I note this openly below.

W_{G} (gain, 50). The main positive term: a unit of cleanliness gain at the end of the trajectory adds 50 to $J(\tau)$. Cedrim et al. [36] found that refactoring often makes structural metrics no better or even slightly worse, which means the score should reward improvement explicitly rather than assume it. Paixão et al. [18] model the trade-off between disruption and improvement as something developers actively manage, which sets the size of this positive term in relation to the penalty terms below.

W_{B} (broken, 28). The largest single penalty term. Paixão et al. [18] show that developers care a lot about avoiding disruptive intermediate states - they are willing to give up roughly a quarter of the available metric improvement to avoid them. The weight is *time-weighted* rather than binary: $r_b(\tau) = b_{\text{ms}}/e_{\text{ms}}$, so a brief broken state is penalised proportionally less than a long one. The numerator b_{ms} counts only wall-clock time during which the build or tests are actually failing - not total refactoring time, and not time the developer spends thinking between changes. A developer who pauses to think carefully before making a clean change is not penalised; a developer who leaves the build broken for five minutes is. Time-weighting also makes the broken term robust to the choice of granularity. The state/action/trace formalism in §3.1 leaves what counts as a "state" flexible - a state can be a commit, an IDE event, or a developer-defined checkpoint - and a binary "was this state broken" counter would change by a large factor across those choices (count once per commit vs. once per keystroke and the broken-state count looks very different). Wall-clock time spent broken does not depend on the sampling rate, so $r_b(\tau)$ stays meaningful at any granularity. The time-weighting choice also lines up with Gautam et al.'s 2025 RefactorBench analysis [9]: their failure-mode #2 shows that strictly rejecting any intermediate broken state hurts LM-agent performance, because some broken intermediates are genuinely necessary for multi-file edits. The anti-gaming rule means W_{B} has to outweigh the single-step gain $W_{\text{G}}\,\Delta C$ for any plausible ΔC.

W_{ST} (skip-tests, 14). Penalises combined refactoring windows (60-second batches) that finish without any test run in between. The 60-second cut-off comes from Murphy-Hill et al. [5]: their study of how developers actually refactor shows that tests are run per batch of changes, not after every single step. I set the weight at half W_{B}: skipping tests is weaker evidence of process damage than an observably broken state, but it is still a signal the score should track.

W_{MI} (manual-when-IDE, 11). Penalises steps where the developer refactored manually when the IDE could have applied the same transformation automatically. Mens & Tourwé [4] note

that IDE refactoring tools run static precondition checks that hand edits may bypass, making the automated path more reliable. Negara et al. [37] find that manual refactorings remain common nevertheless: across 23 developers and 5,371 refactorings, developers frequently performed them by hand even when an automated equivalent was available. Vakilian et al. [38] show that automated refactorings can also be misused, which motivates a graded penalty rather than a binary one. No published study (that I am aware of) directly measures the fault-rate gap between manual and IDE refactoring, so the weight is not fitted to data; it sits within the severity ordering set out above. The ablation in §5.1.2 shows that `manualIde` sits in the top-three single-knob contributors to session-wide divergence-point magnitude, alongside `length` and `broken` (Table 5.5), and the sensitivity sweep in §5.1.1 indicates that the ranking is robust to perturbations of this weight.

W_{L} **(length, 11).** A positive term rather than a penalty: it increases $J(\tau)$ for alternative trajectories that reach the same end state as the user in fewer steps. Harman & Tratt [14] and Mohan & Greer [15] treat refactoring count as an objective on equal footing with quality in multi-objective search-based refactoring; Cortellessa et al. [25] continue that line of work, trading effort against code-quality objectives explicitly. The user trajectory is not penalised for length: a longer path may reflect a larger task rather than inferior process, and treating length as a penalty would conflate task size with process quality. The bonus applies only when the synthesiser supplies a shorter alternative that still reaches the same end state.

W_{CG} **(commit-gap, 7).** Penalises long runs of green checkpoints (refactor passes + tests pass) without a commit in between. The term is based on work on decomposing composite changes. Tao & Kim [39] report that separated commits let reviewers inspect changes more effectively in the same amount of time, and Di Biase et al.'s controlled experiment [40] supports the direction of the penalty: decomposed changes produced fewer comments that wrongly flagged sound code as defective (1 rather than 6, $p = 0.03$). Tangled commits also make later defect attribution less reliable [41]. The claim is deliberately narrow: Kudrjavets et al. [42] find no correlation between PR size and time-to-merge across 1.25M PRs in ten languages, so smaller commit gaps are treated as improving review *comprehensibility* and *rollback locality*, not merge speed. Since no published study (that I am aware of) directly links commit cadence *during refactoring* to rollback or review cost, the weight is not fitted to data; it sits within the severity ordering above. The ablation results support retaining it as a secondary hygiene term: `commitGap` recovers non-zero session-set magnitude on its own (sum/sum 0.126), and removing it changes the ranking (mean $\tau = 0.926$) while leaving most total magnitude intact (sum/sum 0.892).

3.4 The cleanliness sub-score

The cleanliness sub-score $C(s)$ in Equation (3.1) combines six per-state quality signals into a single normalised value in $[0, 100]$. The signal set follows the design principle from §2.3: no single quality measure is reliable across contexts, so combining complementary signals reduces sensitivity to the noise of any one tool. Table 3.3 lists each sub-signal, its cited basis, and how it is computed.

Composition rule. Each raw sub-signal is min–max normalised against the main trajectory's range, inverted for "lower is better" measures (cognitive complexity, coupling, duplication, smells). Sub-signals with degenerate ranges or missing data on the trajectory are dropped and the remaining weights renormalised. Alternative-trajectory checkpoint values are clamped to $[0, 1]$ against the main trajectory's range, so an extreme alt cannot shift the main trajectory's normalisation. The composite is a weighted sum, then scaled to $[0, 100]$. Wagner et al.'s Quamoco framework [51] and the QMOOD quality model [52] both use this kind of layered weighted-sum aggregation, which is a defensible choice when there is no calibration data to fit the weights against.

Uniform sub-weights, by deliberate choice. All six sub-weights are set to 1.0 because there is no published calibration data for weighting these signals across refactoring trajectories. Calibrated weights do exist for snapshot-level static quality composites — notably Coleman et al.'s Maintainability Index, whose coefficients were regression-fitted against expert maintainability ratings on industrial systems [53], and QMOOD [52], whose design-attribute weights were tuned against expert assessments — but those calibrations are fitted to absolute metric magnitudes on a

Sub-metric	Weight	Cited basis	Computation source
Cognitive complexity	1.0	Campbell 2018 [43]; Muñoz Barón et al. 2020 [44] (partial validation); Lavazza et al. 2023 [45] (contradicting)	Per-method mean
Coupling (CBO)	1.0	Chidamber & Kemerer 1994 [46]	CK library, per-class mean
Duplication	1.0	Heitlager et al. 2007 [47]; Fowler 1999 [2]	CPD-detected cloned lines on touched files
Readability	1.0	Buse & Weimer 2010 [48] (feature taxonomy only - *not* the trained model)	Five-feature blend, line-weighted
Smells	1.0	Marinescu 2004 [49]; Arcelli Fontana et al. 2016 [21]; Moha et al. 2010 [19]	PMD violation count on touched files
Cohesion (TCC)	1.0	Bieman & Kang 1995 [50]	Per-class mean, dropped if < 2 methods

Table 3.3: The six sub-signals composing the cleanliness sub-score $C(s)$. All weights are uniform 1.0 following Laplace's principle of insufficient reason (see composition rule below); the production weight bundle in the implementation reflects this.

single snapshot, not to normalised deltas across a sequence of checkpoints, so they do not transfer to the trajectory-relative signals used here. In the absence of trajectory-level calibration data, equal weighting follows Laplace's principle of insufficient reason. The sensitivity experiment in §5.1.1 supports this choice: perturbing the cleanliness sub-weights reshuffles only a small fraction of divergence-point rankings, an order of magnitude fewer than the process-score perturbations.

Caveats deliberately disclosed. Two of the sub-signals have weaker empirical backing than the others. Both are detailed below:

- *Readability.* I use Buse & Weimer's feature taxonomy [48] (line length, indentation, identifier characteristics) as a feature blend, but *not* their trained readability model. Calibrating their trained model to the trajectory-evaluation setting would need domain-specific data I have not collected. The feature blend without the trained model is a weaker signal but it is still defensible as one of six components.
- *Cognitive complexity.* Cognitive complexity is empirically contested. Muñoz Barón et al. [44] re-analysed 24,400 human understandability ratings and partially validated Campbell's claim, finding significant positive correlation with comprehension time and subjective ratings; correlation with correctness was weak. Lavazza et al. [45], on the other hand, find that cognitive complexity predicts understandability about as well as traditional size and cyclomatic-complexity measures, with negligible additional explanatory value. I include cognitive complexity as one of six equal signals, given the mixed evidence around it.

3.5 The divergence-point detector

A *divergence point* is a step in the user's trajectory where an alternative trajectory exists that scores strictly better than the user's under the process score. The detector emits divergence points of four kinds:

$$\mathcal{K} = \{\textsc{ordering},\ \textsc{manual\_refactor},\ \textsc{rework},\ \textsc{hygiene}\}.$$

Each divergence point has four fields: a *kind*, an *anchor step* (the user step where divergence occurs), a *magnitude*, and per-kind context (the reorder window for ORDERING; the replaced refactoring identifier for MANUAL_REFACTOR; the originating–terminal step pair, file, scope, and reverted-line count for REWORK; the hygiene sub-kind and stretch length for HYGIENE).

The magnitude is uniform across all four kinds:

$$\text{magnitude} = J(\tau^*_{\text{final}}) - J(\tau_{\text{final}}),$$

where τ^*_{final} is the alt's process score *rolled forward through the user's post-convergence activity* - i.e. the user's trajectory with the alt's path substituted in at the divergence and the rest of the trajectory unchanged - and τ_{final} is the user's actual trajectory-final score. The comparison is therefore

at the user's trajectory-final state on both sides, making the magnitude directly comparable across all kinds: *"if you had taken this one different path at the divergence point and continued with the rest of your trajectory unchanged, by how many process-score points would your final score have moved?"*

For REWORK specifically, the magnitude is non-trivial even though the alt reaches the same end state as the user: the alt has *fewer* steps because the round-trip work is omitted, so the W_L step-savings bonus contributes positively to the alternative trajectory's score. The reverted-line count is still attached to the divergence point as a separate field (`reworkLineCount`) for explanatory use on the dashboard, but it is not directly used in the ranking magnitude.

Before emitting a divergence point, the detector runs kind-specific filters. Three threshold constants gate detection: MIN_REWORK_LINES = 2 drops single-line REWORK noise; MIN_COMMIT_GAP = 6 requires at least six green-refactor checkpoints between commits before COMMIT_GAP fires; and the 60 s composite-window boundary [5] defines the refactoring batch boundary used by both ORDERING and HYGIENE. The constant 6 is derived from Murphy-Hill's batching heuristic rather than empirical calibration. Since the 60 s composite window already groups tight refactor sequences, setting the threshold at 6 ensures a stretch of consecutive green checkpoints represents a real no-commit gap, not just tight in-batch activity. No published study calibrates this particular threshold, so it is presented as a tunable parameter; the sensitivity analysis in §5.1.1 tests the associated weight W_{CG} rather than the threshold itself.

3.5.1 ORDERING detection

ORDERING detection works on bracket-validated windows of consecutive refactoring steps (§3.10). For each window, the synthesiser enumerates valid permutations of the steps — only those whose terminal AST matches the user's, established by the audit in §3.6 — and computes the process-score delta of each permutation against the user's. One divergence point is emitted per window at its terminal step; the divergence point's magnitude is the best delta across the window's admitted permutations, and all admitted permutations are attached to the divergence point as alternatives. Permutations whose delta is ≤ 0 are retained on the divergence point for informational use (the dashboard surfaces them so the user can inspect the search space), but downstream analyses — the divergence-detection experiment of §5.2.2 and the "beats user" counts reported with it — apply a strict magnitude > 0 filter so only divergence points whose best alternative actually improved on the user's trajectory are counted.

3.5.2 MANUAL_REFACTOR detection

A MANUAL_REFACTOR divergence point is emitted at any user step where (i) the action was performed by hand, and (ii) the resulting diff matches the structural pattern of an IDE-supported refactoring. The pattern match is established by RefactoringMiner-style mining of the diff [7]: if a non-null IDE-mappable spec is recovered from the manual edit, the step is a candidate. Adjacent candidate steps that share the same `(fromSha, toSha)` bracket are grouped into a single composite; the synthesiser (§3.7) builds the alt by applying the recovered spec via the IDE-equivalent pathway (engineering details in §4.6).

3.5.3 REWORK detection

REWORK detection looks for pairs of changes within a trajectory where content is added and later removed (or removed and later re-added) such that the two halves net to a no-op, modulo whitespace. The detector parses each step's unified diff into hunks, resolves each hunk to its enclosing method or class scope, content-hashes the affected lines (whitespace-normalised), and greedily pairs add/remove hunks that share the same `(file, scope, content-hash)`. Each paired `(originating step, terminal step)` becomes a REWORK candidate; pairs below the MIN_REWORK_LINES floor are dropped as line-noise.

3.5.4 HYGIENE detection

HYGIENE detection has two sub-kinds. TESTS_SKIPPED fires once per composite window (60-second batch [5]) that contains at least one refactoring step and no successful test run in between. COMMIT_GAP fires once per stretch of ≥ 6 green-refactor checkpoints (checkpoints whose tests

passed and whose refactoring applied cleanly) without an intervening user commit. Both sub-kinds emit one divergence point per finding.

3.6 Reorder synthesiser

The reorder synthesiser builds alternative trajectories by reshuffling the user's refactoring steps into a different order while still reaching the same end state. Correctness is established in six steps: window-splitting on validator outcomes, dependency-DAG construction over the program entities each step reads and writes, single-static-assignment (SSA) versioning across renames, topological enumeration, depth-first search over a prefix trie of candidate permutations, and a terminal AST-equivalence audit. The engineering choices that make the search tractable - single borrowed worktree, single JDT batch session, git-checkout backtracking - are described in §4.6.

Window splitting on validator verdicts. Only bracket-validated windows (§3.10) are eligible for reordering. The validator returns one of three non-valid verdicts for brackets the synthesiser engine cannot reproduce: UNTYPED when RefactoringMiner cannot map the user's edit to a typed JDT refactoring spec (the typical case for unstructured manual edits); REFACTOR_FAILED when JDT attempts the spec but errors out; and AST_DIVERGED when JDT applies the spec successfully but the resulting AST does not match the user's. Any of the three disqualifies the containing bracket, which is then treated as individual steps that cannot be reordered. The wrap-and-patch layer (§4.6) closes the most common JDT-IntelliJ divergences before this check runs, expanding the set of brackets that pass validation.

Dependency-DAG construction. Each refactoring step carries a structured *spec* describing what entities it reads, writes, creates, or deletes. The synthesiser builds a dependency graph where an edge from step u to step v means v depends on u's effects. This graph defines the constraints on any valid reordering.

SSA versioning across renames. If a method is renamed at step t from `foo` to `bar`, it shows up in later steps under its new name, which breaks simple name-based equality across the trajectory. The synthesiser threads version identifiers through the DAG using single-static-assignment-style versioning: each rename produces a fresh version, and subsequent steps that target the renamed entity bind to the right version. This handles trajectories like Extract Method followed by Rename Method on the extracted method.

Topological enumeration. The synthesiser enumerates topological orderings of the DAG via recursive backtracking. Each enumeration is a candidate permutation of the window's steps. In practice the DAG sizes are small (most brackets contain 2–8 steps), so the enumeration terminates quickly.

Depth-first search over a prefix trie. The enumeration is implemented as a depth-first walk over a prefix trie of candidate permutations, so orderings that share a prefix reuse the project state across that prefix rather than re-applying it from scratch. The engineering of the walk — the borrowed worktree, the JDT batch session, and the git-checkout backtracking that powers the back edges — is in §4.6.

Terminal AST-equivalence audit. For each permutation that reaches a terminal state, the synthesiser admits it only if its terminal AST is equivalent to the user's modulo syntactic differences that do not change behaviour. This is the correctness guarantee: by construction, an admitted reorder reaches the same end state as the user. The hashing mechanism that decides equivalence is described in §4.6.

The valid permutations are then scored under Equation (3.1). The window collapses to a single ORDERING divergence point at its terminal step; the best delta becomes the divergence point's magnitude and every admitted permutation is attached to it as an alternative. Permutations whose delta is ≤ 0 are retained for the dashboard's informational view of the search space; the strict magnitude > 0 filter that picks out divergence points whose best alternative beats the user is applied downstream at the experiment stage (§5.2.2).

3.7 Manual-refactor synthesiser

The manual-refactor synthesiser builds an alternative trajectory in which the user's manual refactoring is instead applied via the IDE's automated pathway. It applies each detected spec via its IDE-equivalent operator and then runs a residual three-way merge to layer the user's non-refactoring edits (for example, unrelated bug fixes interleaved with the refactor) on top of the synthesised refactor. The resulting tree is committed to a shadow repository. The IDE-pathway application itself — worktree borrow, JDT batch session, wrap-and-patch for engine divergences — is described in §4.6.

The residual merge ensures the alt trajectory keeps all the user's non-refactoring edits. This way both trajectories end at the same code state, so the comparison measures the refactoring method (hand vs IDE), not whether the final states match. Without the residual merge, the alt would diverge on non-refactoring changes and the process-score comparison would be meaningless.

3.8 Rework synthesiser

The rework synthesiser builds an alternative trajectory in which the originating–terminal hunk pair identified by REWORK detection (§3.5.3) is skipped. For each pair, the synthesiser replays the [originating, terminal] window as a sequence of surgical patches that omit the reworked chunk while preserving every other change in the window. The operation is atomic per trajectory: either all patches apply, or the alt is thrown away — a partial replay never produces a half-baked alternative. The engineering that makes this work — per-file line-drift tracking, dropping checkpoints whose only change was the excised rework, and folding whitespace-only residue into the preceding checkpoint — is described in §4.6.

The surgical replay preserves all non-reworked content. Because the alternative trajectory skips the round-trip work, it reaches the same end state as the user in fewer steps; the W_{L} step-savings bonus therefore contributes positively to the alt's trajectory-final score, which drives the divergence-point magnitude (§3.5). The reverted-line count is preserved on the divergence point as a separate field (`reworkLineCount`) for purely explanatory use on the dashboard.

3.9 Hygiene synthesisers

The hygiene synthesiser builds alternatives where the user ran tests or committed at points where they didn't, without changing the code itself. There are two sub-kinds. For TESTS_SKIPPED, the alt represents a trajectory in which the user ran the test suite at the anchor checkpoint; the synthesiser injects a synthetic `TEST_RUN_FINISHED` event at that checkpoint but does not touch the test outcome (`tests.success` or `tests.wasSkipped` are left as the user recorded them). The counterfactual is "you should have run the tests here", not "they would have passed if you had": W_{ST}, which is gated on whether tests were run, correctly lifts; W_{B}, which is gated on the broken/passing outcome, is untouched. For COMMIT_GAP, the alt represents a trajectory in which the user committed at the anchor checkpoint, breaking what would otherwise have been a long stretch of uncommitted refactor checkpoints.

Both alter the process metadata at the anchor checkpoint, not the code—because hygiene is fundamentally a process property (when did tests run, when did commits happen). The alt's code remains unchanged; only the metrics shift. By rescoring the trajectory with adjusted metadata and comparing final scores, we measure the cost in process-score points of the skipped hygiene event.

The metadata-adjustment approach is the honest representation of what a hygiene-improved alternative trajectory actually looks like: a developer running tests at a specific checkpoint, or committing earlier, does not change the code at that checkpoint, only the metrics after that checkpoint.

3.10 Validator and behaviour preservation

The bracket-level validator checks that replaying the user's specs in order reaches the user's final state. For each `(fromSha, toSha)` bracket, it replays the specs and verifies that (i) the set of `.java` files changed matches `git diff fromSha..toSha`, and (ii) the terminal AST hash of each

file matches the user's. Brackets that pass both checks are admitted as VALID and become reorder windows; those that fail are treated as individual steps that can't be reordered. The replay reuses the same worktree-borrow and JDT-batch-session machinery as the reorder engine (§4.6).

The validator's admission rate is limited by JDT-IntelliJ engine differences (detailed in §4.6): when the wrap-and-patch layer covers a divergence, the bracket is admitted; when it doesn't, the bracket fails validation even though the refactoring was correct. Covering all JDT-IntelliJ structural differences is future work and is the main reason ORDERING recall isn't higher (reported in §5.2.2).

3.11 Rater design

The annotated sessions used to evaluate the divergence-point detector in §5.2.2 were annotated for multiple divergence kinds in $\mathcal{K} = \{\text{IDE\_REPLAY}, \text{REWORK}, \text{HYGIENE}, \text{ORDERING}\}$. Each session carries a row in `fixtures/sessions/manifest-v2.csv` whose `expected_kinds` cell holds a semicolon-separated subset of $\mathcal{K}$. A session may legitimately exhibit multiple kinds—for example, both an ordering divergence and a hygiene flag at the same step—and the manifest captures all of them.

To check the schema against someone other than the author, two independent annotators (denoted P1 and P2 throughout the results chapter) labelled all 45 sessions against the same schema. The briefing protocol was as follows. Each rater received the schema definition above plus the session artefacts for every recording — the raw event log `events.jsonl` and the wrapping session metadata `session.json`. They did not receive the playbook prompts that drove each recording, the author's `manifest-v2.csv` labels, the per-session `analysis-report.json` produced by Phase A, or any view of the dashboard. The raters read the sessions without seeing the playbook prompts or the detector's analysis, ensuring an unbiased assessment.

Per-kind agreement is reported using Cohen's κ [54], applied separately to each kind in $\mathcal{K}$. For each kind k, the rater's label is treated as the binary judgement "$k \in$ `expected_kinds` for this session", and the resulting 2×2 confusion matrix yields a single κ per rater pair. The Landis and Koch thresholds [55] are used for interpretation: $\kappa < 0.20$ slight, 0.21–0.40 fair, 0.41–0.60 moderate, 0.61–0.80 substantial, 0.81–1.00 almost perfect. With only 45 sessions, the raw agreement counts (yes/yes, no/no, and disagreements from each 2×2 table) are reported alongside κ, since these cell counts are small enough to be informative on their own.

When the two raters disagree with the author on a particular kind, the disagreement is reported as a finding rather than edited into the manifest. The author's labels are not changed post-hoc, and neither are the raters' labels. When both raters agree with each other but disagree with the author (as happens for rework on session 043), those sessions are named explicitly in the results chapter, with a note that the author's labels may be the outlier.

The rater study results—per-kind κ values, raw agreement counts, and disagreement patterns—are reported in Table 5.8 (§5.2.1).

Chapter 4

Tool Architecture

This chapter describes how the tool is implemented. We walk through the pipeline in order: event capture in the plugin (§4.2), shadow-repo reconstruction and worktree allocation (§4.3), per-checkpoint metrics calculation (§4.4), the Eclipse JDT refactoring engine that powers the synthesizers (§4.5), and the four synthesis strategies for building counterfactual trajectories (§4.6). We also describe the split between offline and replay phases (§4.7) and the dashboard that presents the results (§4.8).

4.1 Architecture at a glance

The tool consists of four JVM modules (`:ide-plugin`, `:analysis`, `:refactoring-bundle`, `:shared`) plus a standalone React/TypeScript dashboard (`dashboard/`):

- `ide-plugin/` – an IntelliJ plugin that records the developer's session as a stream of typed events and uploads it to the analysis server. The user manually starts and stops recording a button in the IDE.
- `analysis/` – the Kotlin pipeline; it ingests sessions, reconstructs a shadow git repo, mines and synthesises trajectories, computes per-checkpoint metrics, and emits an `AnalysisReport`.
- `refactoring-bundle/` – a headless Eclipse JDT distribution wrapped in an embedded Equinox OSGi framework; the analysis module talks to it across the OSGi classloader boundary (details in §4.6).
- `dashboard/` – a React/TypeScript single-page app that renders the `AnalysisReport` (details in §4.8).
- `shared/` – cross-wire types (`Session`, `TraceEvent`, `SessionMetadata`, `FileSnapshot`, `EventType`) used on both the plugin and the analysis side.

Figure 4.1 shows the dataflow at a glance.

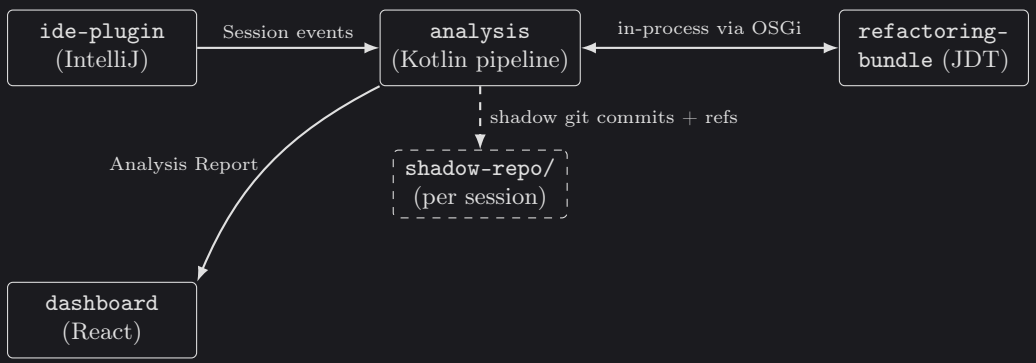

Figure 4.1: High-level dataflow between the recording side (IDE plugin), the analysis pipeline, the embedded Eclipse JDT engine, and the dashboard. The shadow repo holds every reconstructed state – both user checkpoints and synthesised alternatives – as git commits in a per-session repository.

4.2 Event capture

The Intellij plugin is responsible for capturing the Intellij events which are emitted during the user's refactoring process. The captured event types cover editor activity (files opened, closed, saved, modified, renamed, moved), refactoring activity (refactoring start / finish), build activity (build start / finish), test activity (test-run start / finish with pass and fail counts), git activity (one event per new commit observed in the underlying repository), and the session lifecycle (refactoring session start / end).

File modification activity is the noisiest source. IntelliJ emits an event for every keystroke, but we debounce these to avoid noise. A debouncing layer accumulates keystrokes into an *edit burst* across all files: when a 2-second quiet window elapses without further typing, a single burst event is emitted carrying the file's post-edit contents and the structured set of program elements touched.

Although a handful of the fine-grained file-level events (file open, save, and similar) are not consumed by any downstream stage, they are recorded because their capture is essentially free at the plugin layer, and they remain obvious candidates for future work on navigation patterns, file engagement patterns, and read-versus-edit behaviour.

Event normalisation. The captured event stream undergoes three normalisations before downstream processing. First, events are re-sorted by timestamp to reflect actual order. Second, spurious events are removed: FILE_CREATED events that arrive late, and EDIT_BURST events with no actual changes. The result is the *normalised event stream* consumed by every subsequent stage.

4.3 Shadow repo reconstruction and worktree pool

Analysing a user's trajectory requires the ability to reconstruct, and cheaply check out, the exact state of the codebase at any point during the session. This is made possible by the Shadow Repo and Worktree Pool

Shadow repo reconstruction. The shadow repo is built by replaying the normalised event stream into a fresh per-session git repository, one commit per state-bearing event. The baseline commit is the initial source snapshot taken at session start; each subsequent event that carries file changes applies its snapshots to the working tree, stages them, and commits. No-op events – those that carry no file changes, or whose staged diff is blank-line-only – map to the previous SHA without producing an empty commit, so the unique-SHA count is typically much smaller than the event count. The output is a per-event map from event id to SHA, which gives every downstream stage a stable (from-SHA, to-SHA) pair to check out.

Worktree allocation. Borrowers – the validator, the reorder synthesiser, the IDE-replay runner – ask for a fresh worktree at a target SHA, run their operation, and release it back when done. Fresh-per-borrow rather than reuse-across-SHAs is deliberate: it guarantees no leftover build artefacts, stale Gradle cache, or other cross-SHA pollution between consecutive borrows.

4.4 Metrics calculation

For every unique state in the shadow repo the pipeline computes a checkpoint metric record: the build and test outcome, the cleanliness signals (PMD violations, CPD duplications, code readability, and Chidamber–Kemerer structural measures), and the file-level diff against the previous checkpoint.

The build and test outcomes are produced by checking the project state out into a worktree and invoking the project's own Gradle build and test tasks against it, recording for each checkpoint whether the build succeeded and whether the test suite passed. These outcomes are computed for every checkpoint, independent of whether the user actually ran the build or tests at that point in time. This is the stage that dominates the wall-clock cost of the pipeline. Two engineering decisions on the build side brings this cost down: all checkpoints share a single persistent worktree (the only stage that bypasses the fresh-per-borrow rule of §4.3), and the pipeline walks the unique

SHAs in order using detached checkouts to switch between them. The build directory of that worktree is preserved across SHAs, so the Gradle daemon, the Gradle build cache, and incremental compilation all have prior state to reuse on the next checkpoint.

The static-analysis signals are tracked across checkpoints rather than just computed per checkpoint. Each PMD violation and CPD clone group is threaded through the unified diffs so the report can identify when a violation was first introduced, when it was resolved, and which checkpoint transition is responsible. This is what lets the dashboard show, for any refactoring step, the smells it introduced or eliminated. Both user-study participants found this inspection feature valuable during interviews (§5.3.1).

4.5 Applying refactorings via Eclipse JDT

Synthesising counterfactual trajectories requires actually applying refactorings, not just describing them. Building a refactoring engine from scratch was out of scope, so we rely on an existing one. Eclipse JDT was chosen for two reasons. First, operationally: JDT can be linked as a library and driven programmatically from a JVM, whereas IntelliJ's refactoring engine requires launching a full IDE instance via a custom plugin, adding UI dependencies, process boundaries, and startup cost. Second, capability-wise: JDT covers the refactoring catalogue this analysis needs at a maturity level comparable to its use in the Eclipse IDE.

JDT was designed to run inside Eclipse but can be embedded in other JVMs by hosting the Equinox OSGi framework. The Eclipse JDT FAQ [56] distinguishes two deployment modes: bare-classpath (for parser and AST features) and runtime-workbench (for workspace-dependent features). Refactorings require the runtime-workbench mode. The reference implementation is the Eclipse JDT Language Server [57], maintained by the Eclipse Foundation as a headless JDT distribution.

Batch sessions. JDT maintains an indexed project model in memory. Building this model takes roughly a second, so repeatedly rebuilding it between refactorings would be expensive. Instead, the pipeline wraps a whole refactoring window in a single *batch session*: the model is initialized once at the start, reused across all operations, and torn down at the end. Each refactoring then sees a fully indexed model without paying the initialization cost repeatedly.

Anchor resolution. Refactorings need to identify their target, but line numbers shift whenever surrounding code changes. The pipeline addresses this with content-addressed anchors. On the host side, RefactoringMiner's line and column coordinates are used once to locate the AST node, and that node is then summarised as the enclosing type's fully qualified name, the host method's name and parameter-type list, and a canonical hash of the node's own AST subtree. The wire-level spec sent to the JDT bundle carries that anchor; it does *not* carry the original line and column for range-based operations, and for point-based operations it carries them only as a tiebreaker hint in the rare case that two declarations inside the same method hash equal. The bundle finds the host method by name plus parameter types inside the named type, then searches inside it for a node whose subtree hash matches. The same hashing routine is computed on both sides, so the lookup is robust to formatting changes and to line shifts elsewhere in the file: a method that moved from line 42 to line 47 still resolves correctly, as long as its own subtree was not itself rewritten.

4.6 Synthesising counterfactual trajectories

For each divergence point the detector emits, the pipeline synthesises an alternative reference trajectory.

Reorder synthesis. A reorder window of k refactoring steps has up to $k!$ candidate orderings, and each ordering has to be applied against the same starting project state to check whether it reaches the user's final AST (the methodology-level account is in §3.6). The synthesiser does not naively enumerate all $k!$ orderings: the dependency DAG of §3.6 restricts the search to orderings the dependency graph allows, and the remaining orderings are then folded into a prefix trie walked depth-first — each forward edge applies one refactoring, each back edge rolls the worktree to the

parent commit. Two orderings that agree on their first j steps share the first j trie nodes, so each unique prefix is materialised exactly once no matter how many leaves reuse it.

At each leaf of the trie – the terminal state of one full ordering – the synthesiser hashes the touched files' ASTs into a canonical form (§3.6) and compares those hashes against the user's terminal AST hashes, which are precomputed from the user's own trajectory via `git show` at the start of the window (no second worktree is needed). The canonicalisation collapses cosmetic differences (whitespace, comment positioning, formatting variation) and reorderings that do not change behaviour — notably method-declaration order within a class — so two ASTs that are structurally identical produce the same hash. The method-declaration-order canonicalisation matters here because permutations that apply, say, an Extract Method at a different position in the bracket place the new declaration at a different lexical offset, even though the resulting class is functionally identical. An ordering passes if and only if its terminal hashes match the user's. Otherwise, it is discarded.

Three engineering choices make this search possible at user session-scale. First, the whole window runs against a single borrowed worktree, so forward edges, back edges, and the final AST audit all reuse the same on-disk checkout rather than paying the cost of a fresh worktree per ordering. Second, the whole window also runs inside a single JDT batch session (§4.5), so every refactoring inside it sees a warm project model and pays only the per-call cost. Third, back edges in the search roll the worktree back via a detached git checkout and then ask the JDT engine to re-stat its files via the same "files changed under us" pathway an IDE user exercises constantly.

Manual-refactor synthesis and the wrap-and-patch layer. A manual-refactor divergence point names a step where the user performed by hand what the IDE could have done in one operation (§3.7).

Identifying these steps requires first identifying the manual refactorings themselves, which is non-trivial: IDE-driven refactorings arrive pre-tagged in the captured event stream, but a manual refactoring is just a series of edit bursts whose net effect happens to constitute a structured refactoring. The pipeline recovers them by running RefactoringMiner [7] – a standard tool that detects refactorings between two code states by AST-level structural matching – over a sliding window of commit pairs in the shadow repo. The window matters because a manual refactoring may unfold across several intermediate edit bursts, each of which on its own looks like an unrelated change; only by widening the window to cover the full span does the refactoring become visible to the miner. Each detected refactoring is then cross-checked against the captured event stream: a detection whose window contains a matching IDE-emitted refactoring event is recorded as IDE-driven, and one whose window contains no such event is flagged as manual.

Once a manual refactoring is identified, the synthesiser builds the alternative trajectory by applying that refactoring through the JDT engine. A complication which had to be resolved is that the user records sessions in IntelliJ but the pipeline applies refactorings via JDT, and the two engines sometimes produce different terminal ASTs even when they agree on a refactoring's semantics. A wrap-and-patch layer closes this gap between the two engines. Two concrete divergences observed during testing: JDT only adds the `static` modifier to an extracted method when the body requires it, while IntelliJ adds it whenever the body permits; and JDT under-detects outer-scope-variable liveness in conditional branches, producing a void-returning method where IntelliJ would produce a value-returning one. A host-side post-pass detects each pattern and rewrites the bundle's output to match. Where the residual gap cannot be patched safely, the bracket is marked as diverged and not reordered: the wrap-and-patch layer expands the set of allowed brackets but does not fully bridge the two engines. Fully covering all JDT-IntelliJ structural differences remains future work (discussed in §3.10).

Rework synthesis. The detection side (content-addressed hunk pairing) is covered in §3.5.3; what matters here is how the alt is built once a pair is identified. The synthesiser applies a surgical patch at the originating step that skips the round-trip work, and replays the user's intermediate edits on top. A key issue is that those intermediate edits have shifted line numbers around the chunk, so the terminal-step surgery has to translate its line coordinates from the user's tree (where the chunk reappears) to the synthesised tree (where the chunk was never written, so the file is several lines shorter). A per-file drift tracker maintains the divergence between the two trees as a set of zones; each intermediate user hunk shifts every zone past it by the hunk's net line delta, so each zone stays anchored at the same logical content as the user's tree grows or shrinks around it.

After surgery, some of the intermediate checkpoints have no remaining content of their own – their only change was the wasted work the surgery has now excised – and some only contain whitespace changes, such as blank-line edits that surrounded the reworked chunk. Empty checkpoints are dropped from the alt trajectory entirely rather than being committed as no-op steps; checkpoints whose surviving change is purely whitespace are folded into the preceding checkpoint. Both moves shorten the alt's step count, which lifts its process score directly through the step-savings term defined in §3.2.

Hygiene synthesis. Hygiene synthesis has no engineering surface worth describing here: the alt's code is identical to the user's, so synthesis amounts to flipping the relevant process flag on the anchor checkpoint and feeding the trajectory back through Phase B (§4.7). The methodology-level account is in §3.9.

4.7 Pipeline Phases

The pipeline is split into two phases on either side of a single kotlinx-serialisable boundary type, `PhaseAResult`. Phase A covers every stage above – event ingestion, normalisation, shadow-repo reconstruction, refactoring mining, the four synthesisers, the validator, the per-checkpoint metrics computation, and the PMD / CPD cross-checkpoint tracking. It takes on the order of minutes per session, dominated by the per-checkpoint Gradle build and JUnit test runs of §4.4 and the Equinox-hosted JDT applications of §4.5. Phase A emits a `PhaseAResult` containing the trace, the reconstruction, the mined steps and their typed refactoring specs, the per-checkpoint metrics for both the user's trajectory and every synthesised alt SHA, the diff bundle, the PMD / CPD tracking edges, and the reorder trajectories. Phase B takes that record together with a `ScoringConfig` (the six process-score weights of §3.3) and runs the `ReportAssembler`, which calculates and clamps the trajectory's process score to the [0,100] range, attaches alt-vs-user deltas at every checkpoint, and emits the final `AnalysisReport`. Phase B does no filesystem I/O, no JDT work, and no test runs; it is pure in-memory tree-walking and finishes in milliseconds.

This split matters because weights only enter at Phase B: changing them won't break any cached results from Phase A. The sensitivity experiment of Chapter 5 exploits this directly: $12 \times 9 \times 45 = 4{,}860$ scoring configurations are evaluated against the same cached set of `PhaseAResult` dumps in roughly 2.6 seconds of wall-clock time, because each configuration only re-runs `ReportAssembler`. Two Gradle entrypoints make the boundary visible at the CLI layer: `:analysis:phaseA` dumps a `PhaseAResult` JSON to disk, and `:analysis:phaseB` re-runs the cheap second phase against a Phase-A dump and a chosen `ScoringConfig`.

4.8 Dashboard

The dashboard is a React web app that consumes the analysis report and presents the session interactively. It is hosted inside the IDE rather than served as a separate web page: the plugin opens a non-modal IntelliJ dialog containing a `JBCefBrowser`, the IDE's JCEF (Java Chromium Embedded Framework) wrapper, and points it at the React app. Once the page finishes loading, the plugin injects the freshly produced `analysis-report.json` into the page as `window.__REPORT__` and dispatches a `refdash:report-loaded` event; a `useReport()` hook on the React side picks the report up from there. Keeping the renderer inside JCEF means the dashboard is always shown next to the IDE state that produced it (no external server, no cross-origin handshake), while keeping the React + TypeScript developer ergonomics of a normal SPA. Figure 4.2 shows the dashboard in use on one of the user-study sessions.

The trajectory chart plots the user's process score over time, with the synthesised alternative trajectories overlaid as comparison lines so the developer can see how each counterfactual would have scored against their own. Clicking on a refactoring step opens a detail panel showing what the step did, which smells it introduced or eliminated, and which code-quality metrics shifted across it. Each sub-signal of the cleanliness score (PMD violations, CPD duplications, readability, Chidamber–Kemerer measures) is itself click-through: the panel drills down to the underlying per-file readings so the developer can inspect why a step's cleanliness score moved, not just by how much. Divergence points appear as flagged moments on the trajectory, each one a click-through to

the counterfactual that scored higher. A separate panel summarises the highest-impact suggestions for the session as a whole, so the developer has both a per-step view and a session-level takeaway.

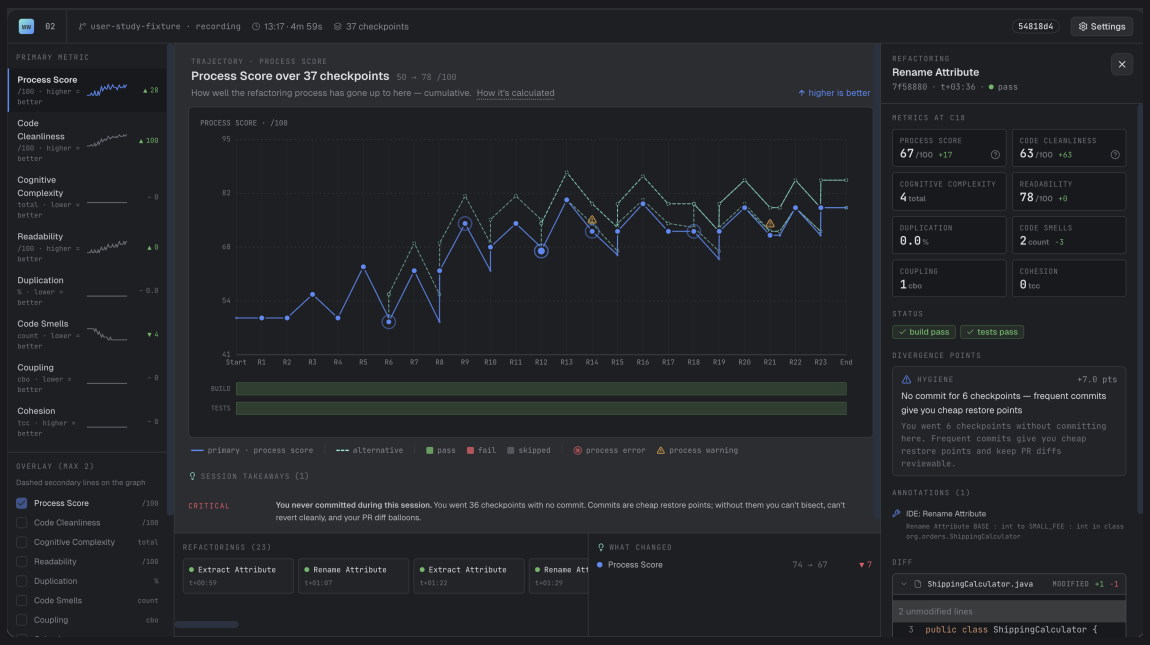

Figure 4.2: The dashboard hosted inside an IntelliJ JCEF window. The trajectory chart (centre) plots the user's process score over the session's 37 checkpoints, with the synthesised alternative trajectory overlaid as a dashed comparison line and divergence-point flags marked along the path. The left rail lists each per-state cleanliness signal with its session-level sparkline; the right rail shows the per-checkpoint detail panel for the currently selected refactoring (a `Rename Attribute`*), including the divergence point it produced, the IDE-event annotation, and the diff of the underlying file. The bottom strip summarises the session's highest-impact takeaway (left) and the per-metric delta across the selected step (right).*

Chapter 5

Experiments and Results

This chapter evaluates the work of the previous two chapters in three parts. §5.1 tests the score formula directly: a sensitivity sweep asks whether the divergence-point ranking is robust to weight perturbations, an ablation sweep asks whether every term in the formula actually contributes, and a single cross-cutting observation reports that reordering refactoring steps rarely helps under the score formula studied here on the current sessions. §5.2 tests the divergence-point detector against ground truth: how well three independent labellers agreed when labelling the same 45 sessions, then per-kind precision and recall against the labelled sessions, then an account of one known recall weakness that is deliberately left open. §5.3.1 reports two short studies in which the tool is used by people other than the author: a 12-session user study with two participants on a new fixture, and a 6-session comparison run on the same fixture by an automated coding agent.

The 45-session injection set. The primary test data is a set of 45 refactoring sessions I recorded by hand. Each session follows a scripted playbook scenario that deliberately induces one specific "bad" refactoring behaviour — a step ordering the playbook author intended as worse than its alternatives, an undo-redo cycle on the same chunk of code, a skipped test cadence, a manual edit where an IDE refactor would have worked, and so on — against a single Java library-style fixture in IntelliJ. The playbook covers all four divergence kinds, with 5 ordering injections, 9 rework, 13 hygiene, and the remainder split between IDE-replay injections and clean-control sessions where no bad behaviour is induced.

Each session carries two distinct labels, used for different parts of the detector evaluation. The first names the deliberately-injected divergence kind — what the experiment was designed to trigger. This is the signal the *injection prominence* claim of §5.2.2 is computed against. The second is a separate hand-label naming the full set of divergence kinds the session should fire, computed independently and allowed to name multiple kinds where a session legitimately exercises more than one. This is the signal the per-kind *precision and recall* claims of §5.2.2 are computed against, and is also the label set whose inter-rater agreement is reported in §5.2.1. P1 and P2 produced their own independent labelling for all 45 sessions before either performed a user-study session of their own.

Reporting conventions. Per-cell counts across the four kinds and 45 sessions are small, so raw counts are reported alongside percentages everywhere they appear. Every magnitude number in this chapter is the trajectory-final J delta defined in §3.2, uniform across all four divergence kinds.

5.1 Testing the score formula

This section evaluates the process-quality score J directly: how sensitive its divergence-point ranking is to the choice of weights (§5.1.1), how much each individual term contributes to the ranking (§5.1.2), and an overall finding about what the formula does and does not detect on the session set (§5.1.3).

5.1.1 Sensitivity sweep

The first experiment asks how the divergence-point ranking responds to perturbations of the weights in the process-score formula. If tweaking a weight significantly reshuffles the ranking, then the ranking depends on choices we cannot justify. The experiment runs against three session sets and uses two perturbation approaches. The first is a single-knob sweep that varies one weight at a time. The second is a multi-knob Monte Carlo that draws every weight simultaneously from a log-normal distribution centered around the production weights. The headline result is top-1 stability across all three session sets and both approaches, reported below alongside the places where the formula does respond to weight perturbations.

Design. The single-knob sweep fixes every weight in `ScoringConfig.PRODUCTION` except one, scales the chosen weight by one of nine factors in $\{0.1, 0.25, 0.5, 0.75, 1.25, 1.5, 2.0, 4.0, 10.0\}$, reassembles the analysis report, and compares the perturbed divergence-point ranking against the production baseline. There are twelve weights (6 process-side and 6 cleanliness-side) and nine factors per weight, so the design produces $12 \times 9 \times |\mathcal{C}|$ rows per session set $\mathcal{C}$. The multi-knob setup draws 200 independent samples per fixture. Each weight W_i is scaled by $\exp(\sigma \cdot Z_i)$ with $Z_i \sim \mathcal{N}(0, 1)$ drawn independently per weight, and $\sigma = \ln 2$, so around 95% of samples land inside $[\times 0.25, \times 4]$ on each weight, and the distribution is symmetric in log-space.

The experiment is run against three session sets: the 45 hand-labelled injection sessions of §5.2.2, the 12 user-study sessions of §5.3.1, and the 6 agent sessions of §5.3.2. These differ in their per-fixture divergence-point profiles. The injection sessions have at most 5 divergence points per fixture, 66 in total, and concentrates on single-kind exercises. The user-study sessions have up to 8 per fixture across naturalistic sessions, 50 in total. The agent sessions hves up to 3 per fixture across structurally simpler sessions, 10 in total.

Three ranking-stability statistics are reported per row. Kendall's τ-b [58, 59] is a rank correlation coefficient designed for short rankings on small samples, reported as the percentage of rows where $\tau = 1.0$ (ranking unchanged) and separately as the percentage where $\tau < 0$ (ranking inverted). Top-N hit rate is the fraction of the production top-N that survives in the perturbed top-N, reported at $N = 1, 3, 5$. Top-1 is the only metric that works across all three session sets, since it bites on any fixture with at least one divergence point. Top-3 and top-5 are reported on the user-study sessions only, where per-fixture sizes are large enough to make them informative; on the other two session sets every fixture has at most 5 divergence points and the top-5 comparison collapses to set-preservation [60]. Both metrics require a deterministic ranking, but several divergence-point kinds produce ties: every HYGIENE and COMMIT_GAP point carry the same magnitude (exactly W_{CG}, since the commit-flip alt drops the gap count from one to zero at trajectory-final), and divergence points saturated against the $[0, 100]$ score clamp share a clamp-induced magnitude that is weight-invariant. Ties are broken by ascending step index, applied uniformly across both the production baseline and every perturbed ranking, so the comparison is well-defined.

Single-knob results. Table 5.1 reports top-1 stability across the three session sets. The user-study sessions is the most demanding test: it has the longest per-fixture rankings, the highest divergence-point density, and zero clipped baseline divergence points against the $[0, 100]$ score clamp. The injection sessions shows the highest stability, but only because it has shorter rankings to reshuffle in the first place. The agent sessions sit between them, but 30% of its baseline divergence points hit the score suppression clamp, which suppresses the observable effect of weight perturbations. On the user-study sessions the top-1 recommendation is preserved on 1,239 of 1,296 rows (95.6%).

session set	rows	$\tau = 1.0$	top-1 $= 1$	$\tau < 0$	baseline saturation
Injection (45)	4,860	99.2%	99.9%	0.0%	13.6%
User study (12)	1,296	86.3%	**95.6%**	0.7%	**0.0%**
Agent (6)	648	97.2%	97.5%	2.5%	30.0%

Table 5.1: Single-knob sensitivity across three session sets. The user-study sessions is the cleanest benchmark: longest per-fixture rankings, zero saturated baseline divergence points. Top-3 and top-5 stability on the user-study sessions are 93.0% and 98.1% respectively; top-5 on the other two session sets is trivially 100% because no fixture there has more than 5 divergence points.

Sources of ranking instability. Table 5.2 reports per-knob top-1 disruption counts on the user-study sessions. The pattern is clear. The six cleanliness sub-weights (cognitive, coupling, duplication, readability, smells, cohesion) never disrupt the top-1 recommendation across any of the 108 rows per knob. Every top-1 shift is driven by a process-side weight. The two largest disruptors are the commit-gap weight W_{CG} (19 of 108 rows) and the manual-when-IDE weight W_{MI} (12 rows). The remaining process-side weights account for between 4 and 9 shifts each, including the cleanliness-gain weight W_{G}, which only disrupts the top-1 at $\times 2$, $\times 4$, or $\times 10$. This decomposition tells us the score formula's *effective parameter dimension* is closer to four or five than to twelve. The cleanliness score doesn't change the ranking because it adds the same amount to both paths, and the process-side weights do the rank-shaping.

knob	top-1 disruptions / 108 rows
`commitGap`	19
`manualIde`	12
`length`	9
`skipTests`	8
`gain`	5
`broken`	4
all six cleanliness sub-weights:	*0 disruptions*

Table 5.2: Per-knob top-1 disruption counts on the user-study sessions. Cleanliness sub-weights never reshuffle the highest-magnitude divergence point in the sweep; the process-side weights account for every top-1 shift.

Multi-knob Monte Carlo. The single-knob sweep only tests one weight at a time. The Monte Carlo approach perturbs every weight together: 200 samples per fixture, with each weight drawn independently from $\exp(\ln 2 \cdot Z)$ for $Z \sim \mathcal{N}(0, 1)$. Table 5.3 reports the distribution of τ and top-N hit rate across $200 \times |\mathcal{C}|$ joint-perturbation samples per session set.

The multi-knob result is meaningfully weaker than the single-knob headline. On the user-study sessions, the top-1 recommendation changes on 19% of joint-perturbation samples, against 4% under single-knob, and mean τ falls to 0.781. The injection sessions are much more stable at $\tau = 0.986$ mean, but again only because their per-fixture rankings are too short to allow much reshuffling. The agent sessions sits between them on τ, and its top-3 stability is trivially 100% because every agent fixture has at most 3 divergence points. In short, the formula is robust to small joint perturbations and degrades smoothly with larger ones. It is *not* robust to arbitrary combinations of weights, even within the realistic range we tested.

session set	samples	mean τ	top-1 = 1	top-3 = 1	top-5 = 1
Injection (45)	9,000	0.986	99.8%	97.2%	100.0% †
User study (12)	2,400	**0.781**	**81.2%**	**75.1%**	92.8%
Agent (6)	1,200	0.736	86.1%	100.0% †	100.0% †

† trivially 100%: no fixture in that session set has $> N$ baseline divergence points.

Table 5.3: Multi-knob Monte Carlo robustness across three session sets (200 samples per fixture, log-normal $\sigma = \ln 2$). The user-study sessions are the discriminating benchmark; the multi-knob top-1 stability is ~ 14 percentage points lower than the single-knob result on the same set.

Saturation and score-clamp bias. One concern is whether the τ and top-N numbers reflect the $[0, 100]$ clamp on the process score rather than true stability. When both user and alt scores hit the ceiling, their magnitude is zero regardless of weights, which would inflate τ. Saturation is therefore tracked per fixture. The right-hand column of Table 5.1 shows the three session sets differ substantially. The injection sessions has 13.6% saturated baseline divergence points, concentrated on seven short fixtures. The agent sessions has 30.0%, because most agent sessions push the score close to the ceiling on the easier playbook sessions. The user-study sessions has *zero* saturation. This is the main reason for leading with the user-study sessions: its τ and top-N values are clean perturbation responses, not clipping artifacts, and the multi-knob top-1 = 81% figure isn't explained by saturation.

Caveats and scope. Three main limitations. *First*, the experiment establishes that the formula is stable near the production weights. It does not establish that the production weights themselves are uniquely correct, only that small changes to them mostly preserve the ranking. To properly validate this, we'd need to refit the weights against an external outcome dataset, e.g. longitudinal code-quality measurements, but no such dataset exists for this problem. *Second*, the multi-knob test is centered around the production weights and tests the neighborhood within roughly $\pm 2\sigma$ (about $\times 0.25$ to $\times 4$). The formula visibly degrades on the user-study sessions even inside this range ($\tau = 0.78$ mean), so the robustness claim is local rather than global. *Third*, the user-study sessions are $n = 12$ sessions across 2 participants; we haven't tested it on a broader set of naturalistic sessions. We document these limits as threats to validity in §5.2.3 and the discussion chapter. The chapter does not claim more than the experiments support.

TODO: MOVE ABOVE TO VALIDITY SECTION

5.1.2 Ablation sweep

The sensitivity sweep shows the ranking is robust to weight perturbations. This experiment asks the different question of whether every term in the score formula carries substantial weight. If removing any one term left the ranking unchanged, that term would be decorative, and if zeroing every process term simultaneously did not collapse the divergence-point magnitudes to zero, then some weight-independent signal would be doing the work and the formula would just restate that signal. A full power-set sweep across the six process-side weights ($2^6 = 64$ variants per fixture, 2,880 rows total) rules out both possibilities: sum-of-magnitude recovery rises monotonically with the count of active terms from 0.000 (every process term zeroed) to 1.000 (full formula), and mean τ rises in parallel from 0.830 to 1.000.

Design. The experiment enumerates every subset of the six process-side weights, testing each subset by keeping those weights at production and zeroing the rest. With six weights the full $2^6 = 64$-variant power set fits comfortably and runs in roughly 1.7 seconds of wall-clock time. This approach draws from two research communities. Composite-indicator sensitivity analysis [61] establishes systematic weight perturbation as standard practice for composite indices. Shapley-value feature attribution [62, 63] uses enumeration over all 2^n subsets and is exact rather than approximate when n is small; sampling approximations such as KernelSHAP only become necessary once n becomes large and enumeration becomes intractable. With $n = 6$ here, a full enumeration is the appropriate choice. The cleanliness sub-weights are kept at production across all variants because the sensitivity sweep already showed cleanliness perturbations almost never reshuffle the ranking; ablating them would inflate the variant count from 64 to 4096 for minimal new information.

Two recovery statistics are reported. Mean recovery is the per-fixture ratio of perturbed total absolute magnitude to the baseline total absolute magnitude, averaged across fixtures, and is shown for consistency with earlier analysis. Sum-over-sum recovery is the sum of |magnitude| across every divergence point in every fixture, divided by the same sum at production, and is the cleaner measure of the fraction of the sessions' total magnitude that survives a given ablation. Sum-over-sum is better than per-fixture mean because it avoids the $0/0 = 1.0$ problem on empty fixtures. We treat it as our main metric.

Sanity check. The full-formula variant (all six terms active) reproduces production exactly: $\tau = 1.000$, recovery $= 1.000$, and mean $|\Delta\text{magnitude}| = 0.000$ on every fixture. Comparison and production use the same system.

Monotone by active-term count. Table 5.4 groups the 2,880 rows by how many process-score terms are active. Both τ and sum-over-sum recovery increase with each added term, with no jumps. The key finding is when all terms are zeroed (active-count $= 0$): the total magnitude across all 45 fixtures drops to zero. This means there's no baseline magnitude independent of the weights; every term does real work, and removing all six leaves nothing behind. (The mean-recovery value is 0.356 at that point, which is why we use sum-over-sum instead.)

Per-term single-knob recovery. Table 5.5 shows how much magnitude each term contributes when it's the only active term. Three observations follow. The `length`, `broken`, and `manualIde` terms are the strongest contributors, each recovering roughly a third of the total; these match the

active terms	mean τ	mean recovery	sum/sum	n
0 (all stripped)	0.879	0.356	**0.000**	45
1	0.901	0.490	0.210	270
2	0.922	0.606	0.398	675
3	0.943	0.710	0.568	900
4	0.963	0.810	0.725	675
5	0.982	0.907	0.869	270
6 (full)	1.000	1.000	1.000	45

Table 5.4: Mean τ and magnitude recovery grouped by how many process-score terms are active. Both statistics increase with the number of active terms; sum/sum recovery reaches 0 when all terms are zeroed.

values in Table 3.2 (§3.3), connecting methodology to results. The `gain` term alone recovers 0.000 despite having the largest weight (50), because gain relies on the cleanliness delta ΔC, which is near zero when user and alternative reach the same state; gain has no effect alone but matters in combination, as Table 5.4 shows. Finally, the gap between `commitGap` (0.096) and `length` (0.389) shows that recovery depends on both weight and how much the signal varies across sessions, not solely weight alone.

term	mean τ	mean recovery	sum/sum
`length`	0.879	0.654	**0.389**
`broken`	0.923	0.614	0.358
`manualIde`	0.879	0.439	0.311
`skipTests`	0.879	0.467	0.106
`commitGap`	0.967	0.414	0.096
`gain`	0.879	0.356	**0.000**

Table 5.5: Per-term single-knob recovery (each term active alone, others zeroed). The `manualIde`, `broken`, and `length` terms each recover roughly a third; `gain` alone recovers 0.000 *because the cleanliness delta is near zero on most fixtures, so it has no effect when isolated.*

Per-term leave-one-out recovery. Table 5.6 shows what happens when you remove one term from the full formula. Removing `length` causes the largest drop in recovery (0.700), so the step-savings bonus from reorder and rework contributes more than any other single term. Removing `gain` pushes recovery above 1.000 to 1.113 — the most surprising result: without `gain`, total magnitude is *larger* than at production. The unit being summed here is the per-divergence-point magnitude defined in §3.2, i.e. the trajectory-final $J(\tau^*) - J(\tau)$ delta between each alternative and the user it is compared against; the sum is over |magnitude| across every surfaced divergence point, including the ones whose alternative scored *worse* than the user (negative magnitudes contribute their absolute value). The mechanism behind the inflation is that `gain` $\cdot \Delta C$ is small but typically positive for alternatives, so it partially offsets the penalty terms; remove it and the penalty terms determine the alt-versus-user delta unopposed, producing larger per-DP absolute magnitudes which accumulate in the sum. Were the experiment to filter to alt-beats-user DPs only, removing `gain` would in fact *lower* the total, since alternatives sitting just above the beats-user threshold flip below it without the offset. The `gain` term therefore plays a regularising role against the penalty terms rather than acting as a positive contributor in isolation. Removing `skipTests` or `commitGap` reshuffles the ranking somewhat but moves recovery less than removing `length` or `manualIde` does, so these terms primarily affect *which* divergence point is biggest rather than *how* big.

Cross-set rerun on the user-study sessions. The ablation above runs on the 45 injection sessions, which have few divergence points per fixture (max 5, mean 1.5). Running the same test on the 12 user-study sessions, which have larger rankings (max 8, mean 4.2) and no saturation, produces different results. Table 5.7 compares leave-one-out τ across both sets. On user-study sessions, the most important term is commit-gap (W_{CG}): removing it drops τ from 1.000 to 0.641, roughly twice the drop of any other term. On injection sessions, W_{CG} is one of the smaller-impact terms ($\tau = 0.930$ on removal). The reverse holds for W_{MI} and W_{L}: they drive injection-session ranking but have smaller effects on user-study sessions. This difference matters: *which terms matter depends on the type of sessions*, so the weights have different effects on different session

removed term	mean τ	mean recovery	sum/sum
`length`	1.000	0.814	0.700
`broken`	0.967	0.938	0.775
`manualIde`	1.000	0.895	0.795
`skipTests`	1.000	0.889	0.894
`commitGap`	0.930	0.959	0.939
`gain`	0.993	0.945	**1.113**

Table 5.6: Per-term leave-one-out recovery (five terms active, one removed). Removing ***`length`*** *causes the largest sum/sum drop; removing* ***`gain`*** *inflates sum/sum above* 1.000*, indicating that* ***`gain`*** *regularises against the penalty terms rather than acting as a positive contributor in isolation.*

types. Removing the cleanliness-gain term W_{G} still has no effect on either set, so it acts as a regularizer, not a rank-shaper.

removed term	injection τ	user-study τ
`commitGap`	0.930	**0.641**
`manualIde`	1.000	0.826
`broken`	0.967	0.837
`skipTests`	1.000	0.844
`length`	1.000	0.905
`gain`	0.993	0.992

Table 5.7: Leave-one-out ranking stability against the injection sessions (Table 5.6) and the user-study sessions. The term-importance ordering inverts between the two session sets: W_{CG} is the dominant rank-carrier on the user-study sessions, W_{MI} and W_{L} on the injection sessions.

Caveats. Five key limits apply. The cleanliness sub-weights are not ablated because they had almost no effect (§5.1.1). Saturation has only a small effect on this experiment: the mean saturated-divergence-point count is 0.20 to 0.29 across variants, so saturation doesn't greatly reduce the signal. τ-b is unstable on small divergence-point counts, and many fixtures have one or two divergence points where τ is 1.0; mean τ at active-count = 5 partly reflects this floor, not pure ranking quality. The `gain`-alone row equals the empty baseline on recovery (0.000); the regularizer role shown in the leave-one-out table explains this. Finally, with the earlier line-count approach the active-count = 0 floor was 0.119 because line-count magnitudes were weight-invariant by design; with the trajectory-final approach used here the floor collapses to 0.000. The relative importance of terms is consistent across both approaches.

5.1.3 Reordering rarely helps on independent refactorings

The single most interesting finding is that reordering refactoring steps almost never improves the score on these sessions. The reorder synthesiser tested 194 permutations across the 45 sessions, which collapsed into 24 distinct reorder windows. Of those 24, only 4 had a permutation better than the user's order (16.7%; Table 5.10). On the 5 sessions where the playbook author selected an ordering with the intent of being worse than its alternatives, none produced a better alternative under the score formula — which is itself part of the finding rather than separate evidence: the formula and the synthesiser together cannot distinguish the author-intended-as-worse ordering from any of its permutations.

TODO: DELIBERATLEY BAD ORDERINGS IS WEAK!!!!

The reason is straightforward. The reorder synthesiser works correctly, but most IDE refactoring pairs are independent by design: they don't interact, so every permutation reaches the same final state. This means the final cleanliness metrics are identical across permutations, the gain term has zero ΔC, and intermediate quality doesn't affect the score at all. This isn't a bug—it's a real finding: under the current score design, the order of independent refactorings doesn't measurably matter.

This has theoretical support. Mens, Taentzer and Runge [64] show that refactorings as graph transformations with parallel independence (empty critical pairs) means every permutation reaches the same final state, and IDE refactorings are designed to have this property. Prior empirical work either claims order matters based on interactions between code smells [65] or tests on a single

small fixture without checking AST equivalence [66]. Our 45-session, 194-permutation result is, to our knowledge, the first systematic evidence on real IDE sessions that independent refactorings dominate and that reordering produces zero score improvement in practice.

Three things follow. First, on independent refactorings, ordering matters less than the literature suggests. Developers often assume different orders produce different code, but for IDE refactors on independent targets they don't—the session set confirms this. Second, the detector still found 7 windows where order mattered; these are interesting cases with non-commuting pairs like inline-then-rename versus rename-then-inline, where the rename affects the body being inlined and the final state differs. Third, to make ordering a larger finding, we'd need to deliberately inject non-commuting sequences or add intermediate-state quality to the score formula; the current sessions and score show that independent-reorder changes fall below the detector's threshold by design.

This strengthens the chapter's claim. The detector's job is to surface measurable improvements, and independent refactorings have no measurable improvement to show - the dashboard stays silent or shows zero magnitude.

5.2 Testing the divergence-point detector

This section evaluates the divergence-point detector against external ground truth. §5.2.1 reports the inter-rater agreement that establishes the per-kind labels as a defensible ground-truth signal. §5.2.2 then measures the detector's precision and recall against those labels on the 45 injection sessions. §5.2.3 discusses one known recall weakness in the resulting numbers that is deliberately left open.

5.2.1 Inter-rater reliability

P1 and P2 each labelled all 45 sessions against the multi-label schema in §3.11, working only from `events.jsonl`, `session.json`, and a written copy of the schema. Neither rater saw the playbook prompts, the author's own labels, the divergence-experiment analysis reports, or any dashboard output; the labelling pass ran before either rater performed a user-study session. Table 5.8 reports per-kind Cohen's κ [54] for the three rater pairs against the 45-session set. All four kinds reach at least substantial agreement by the Landis and Koch thresholds [55]. Ordering and IDE-replay land at $\kappa = 1.00$ across every pair: all three labellers agree on every one of the 45 sessions. Rework lands at $\kappa = 0.86$ between the author and each of the independent raters, and at $\kappa = 1.00$ between P1 and P2. Hygiene lands between $\kappa = 0.72$ and $\kappa = 0.86$, depending on which pair is compared.

kind	Author vs P1	Author vs P2	P1 vs P2
ORDERING	1.000	1.000	1.000
IDE_REPLAY	1.000	1.000	1.000
REWORK	0.861	0.861	1.000
HYGIENE	0.848	0.723	0.862

Table 5.8: Per-kind Cohen's κ on the 45-session set between the author and the two independent raters and between the two independent raters. Ordering and IDE-replay are perfect on every pair; rework and hygiene reach substantial-to-almost-perfect agreement on every pair under Landis and Koch's interpretation thresholds.

The two imperfect kinds show specific disagreement patterns. Rework disagreements occur on exactly two sessions out of 45: session 024 (borderline suboptimal-ordering) has a rework label from both raters but not the author, while session 043 (borderline add-then-revert) has a rework label from the author but neither rater. Since P1 and P2 agree with each other on both, their consistent judgment suggests the author's labels on these two sessions may be outliers. Hygiene disagreements cluster on three manual-move-method sessions (014, 015, 016) where P2 added a hygiene label that neither the author nor P1 did; plus session 039 (borderline no-commit-stretch) where the author added hygiene but neither rater did. We report these as findings rather than revise the labels, following our protocol (§3.11) not to edit labels after the fact to artificially boost agreement scores.

5.2.2 Divergence experiment

The third experiment evaluates the divergence-point detector itself against ground truth. For each of the 45 recorded sessions, the analysis pipeline generates a report using the production weights and compares the report's divergence points against the session's `expected_kinds` label (the set of kinds a perfect detector would surface alternatives for) and the implied injection kind from the session's playbook pattern. The output is a 54-column row per session covering per-kind classification (true and false positives and negatives), magnitude statistics, and injection prominence; the full sweep takes roughly 5 seconds of wall-clock time. The headline numbers, expanded in the parts below, are perfect precision on all four kinds (rework, hygiene, ordering, and IDE-replay all at 1.00), and every caught injection ranked at top-1 within its own session.

Precision and recall. We report precision and recall as separate numbers rather than rolling them into F1 or accuracy. The reason is that the two kinds of mistake the detector can make are not equally bad: if the detector shows the user an alternative that doesn't really exist (a precision miss), it actively misleads them and wastes their time; if the detector quietly fails to show an alternative that would have helped (a recall miss), the user is no worse off than if they had never opened the dashboard. Treating these as the same kind of error, the way F1 does, would hide the difference. Accuracy is also unhelpful here because most of the cells in the per-kind confusion matrix are true negatives — a detector that does nothing at all would still score around 75% accuracy on these refactoring sessions, which doesn't tell the reader anything useful.

Table 5.9 reports per-kind precision and recall. Rework and hygiene are perfect on both axes: 9 of 9 rework injections and 13 of 13 hygiene injections are caught at 1.00 precision. IDE-replay is also precision-perfect. Ordering has perfect precision but only 0.36 recall on any-expected (and 0.40 recall on injection-only), because the reorder synthesiser's `splitOnInvalid` validator check (§3.6) disqualifies any window containing a step the synthesiser engine cannot reproduce, and on this session set most ordering-injection windows contain at least one such step. The recall gap is documented and deliberate - its rationale is in §5.2.3. Per-kind inter-rater reliability of these labels, against two independent raters, is reported in §5.2.1.

kind	precision	recall (injection)	recall (any expected)
ORDERING	1.00	0.40	0.36
IDE_REPLAY	1.00	0.76	0.76
REWORK	1.00	1.00	1.00
HYGIENE	1.00	1.00	1.00

Table 5.9: Per-kind precision and recall on the 45-session set. The "injection" recall column counts only the kind the playbook explicitly injected; "any expected" counts any kind the session's `expected_kinds` label includes.

Detection quality. A precision/recall number says nothing about whether the alternatives the detector surfaces are actually useful improvements over the user's own trajectory. Table 5.10 reports the number of divergence points the detector fires per kind, the number of alternatives it enumerates, and the fraction of fires whose best alternative beats the user's trajectory under the production score formula. Across all kinds, best-alt magnitudes range from -5 to $+23$ with mean $+6.39$, so the percentages in the "beats user" column below can be read against an absolute scale of how much each winning alternative actually improved on the user's trajectory.

Hygiene never proposes a useless alternative: every one of its 13 divergence points is a real improvement over the user's trajectory (100%). IDE-replay proposes a real improvement in 12 of 16 fires (75.0%); the four misses break down as three sessions where the user's process score was already pinned at the lower end of the $[0, 100]$ range so neither the user nor the IDE-driven alternative can score below it (the magnitude is mechanically zero rather than evidence about the `manualIde` penalty), and one session where the IDE alt scored 5 points lower than the user's manual edit. The four misses are a property of the score-range floor, not a calibration finding about `manualIde`.

Rework beats the user in 6 of 13 fires (46.2%); one of the seven non-beats is a magnitude-zero no-op (stripping the rework loop did not move the final score) and the other six produce an alternative with negative magnitude. Two readings sit on top of each other here. The first is

implementation: the rework synthesiser is intentionally simple, stripping out the user's add-then-undo edits without modelling what the intermediate states were *for* — if the user added a guard, ran the tests, decided the guard was unnecessary and removed it, the synthesiser treats the whole excursion as wasted motion, but the user's trajectory may have legitimately benefited from the validation step (one more checkpoint, one fewer commit-gap, a test run that pushed cleanliness up). The second is methodological: undoing work is not inherently a bad behaviour. A careful user who paths through a slightly longer trajectory to verify intermediate correctness can, and on these sessions sometimes does, score higher than a trajectory that takes the shortest line to the same final state. The 46.2% beat rate is therefore not evidence that the rework detector is miscalibrated, but evidence that the score formula correctly distinguishes "rework that wasted process signal" (the six beats) from "rework that earned process signal en route" (the seven non-beats) — and that the simple loop-stripping synthesiser cannot tell the two apart by inspection of the diff alone.

Ordering surfaces 24 distinct divergence points from 194 enumerated permutations, but only 4 of the 24 windows (16.7%) have a best permutation that beats the user's order. The 194/24/4 breakdown explains the headline result in §5.1.3: the synthesiser enumerates 194 permutations across the session set, those collapse to 24 reorder windows (one divergence point per from-SHA / to-SHA window, magnitude set to the best permutation in the window), and only 4 of those 24 windows produce any positive magnitude improvement at all.

kind	DPs	alts enum.	beats user	beats fraction	max magnitude
REWORK	13	13	6	**46.2%**	8 points
HYGIENE	13	13	13	**100%**	9 points
IDE_REPLAY	16	16	12	**75.0%**	23 points
ORDERING	24	194	4	**16.7%**	9 points

Table 5.10: Per-kind detection quality. "DPs" is the number of divergence points the detector fires; "alts enum." is the total number of alternative paths enumerated across those DPs (a single ordering DP enumerates up to k! permutations of a k-step window); "beats user" is the number of DPs whose best alternative trajectory beats the user's trajectory.

Injection prominence. The most user-facing claim in this chapter is in Table 5.11: of the 25 injections the detector caught with a positive-magnitude alternative (8 hygiene, 12 IDE-replay, 5 rework), every single one was the highest-magnitude divergence point in its session. The mean rank is 1.00 across the three kinds that produced any catch at all. A user who only opens the top recommendation in the dashboard of §4.8 therefore always sees the deliberately bad behaviour the playbook injected, and no useful alternative of another kind masks the injection. Ordering injections are absent from the prominence table because zero of the 5 ordering-injection sessions produced a positive-magnitude reorder alternative under the production formula — consistent with the score-formula-and-synthesiser insensitivity to commuting reorders reported in §5.1.3 rather than a separate failure of the prominence claim. The prominence number is partly a property of the controlled-injection design, since the playbook injects one dominant bad behaviour per session and that behaviour is structurally likely to be the largest divergence point; the prominence number on uncontrolled real-world sessions would almost certainly be lower. The participants' sessions described in §5.3.1 deliberately omit pre-declared expected kinds, so no prominence number is reported for them; the per-session per-kind distribution reported there is the closest external check on the prominence claim.

injected kind	n caught	@ rank 1	mean rank
HYGIENE	8	8	1.00
IDE_REPLAY	12	12	1.00
ORDERING	0	—	—
REWORK	5	5	1.00

Table 5.11: Injection prominence: when an injection's kind is caught, where does the corresponding divergence point sit in the session's per-magnitude ranking? Top-1 on every catch across all four kinds.

Caveats. Three limitations qualify the divergence numbers. The session set is 45 hand-recorded sessions, so per-cell counts are small (only 5 ordering injections, for example); the per-kind ratios

are directionally correct but not enough to support tight confidence intervals, which is why raw counts are reported alongside percentages throughout. The detector reports each divergence point against the user's recorded step index, but step boundaries are a function of the plugin's edit-burst-flush cadence and are not semantically stable across recordings of the same logical activity; recall is therefore reported under the looser anywhere-in-session policy rather than against the manifest's `target_step` label. The "beats user" computation uses a strict magnitude > 0 threshold against trajectory-final J, so a positive magnitude genuinely means the alternative scores higher under the production formula; hygiene commit-gap divergence points have a discrete magnitude exactly equal to W_{CG} because the commit-flip alt drops the gap count by one at trajectory-final. The 194 enumeration count for ordering is the synthesiser's reach across the session set rather than the user-facing number, which is the 24 surfaced divergence points.

5.2.3 Scoped-out fix: ordering recall

One known weakness in the divergence numbers is deliberately left open. Ordering recall sits at 0.36 on any-expected (equivalently 0.40 on injection-only). The cause is the `splitOnInvalid` validator check of §3.6: any reorder window containing a step the synthesiser engine cannot reproduce (one of the three non-valid verdicts defined there) is collapsed into singletons and contributes no permutations. The gap is left open because §5.1.3 has already shown that the score formula is insensitive to commuting reorders. If the validator check were relaxed, more reorder windows would be admitted, but every newly-admitted window would (by the same commuting-refactor argument) carry magnitude zero. Recall would rise, the fraction of windows that beat the user would fall by the same amount, and the substantive finding of §5.1.3 would be reinforced rather than overturned. Relaxing the check is therefore a cosmetic improvement to the recall number that strengthens the headline claim of §5.1.3 rather than changing it.

5.3 User and agent studies

This section reports two short studies in which the tool is used by people other than the author. §5.3.1 is a 12-session user study with two participants on a new fixture. §5.3.2 is a 6-session comparison run on the same fixture by an automated coding agent.

5.3.1 User study

The 12-session user study steps outside the controlled-injection sessions to two new questions. First, on sessions recorded by independent participants on a different codebase and without the playbook's controlled-injection structure, does the detector's per-kind output remain coherent? Second, do per-session divergence-point rates evolve as participants accumulate tool feedback across a multi-session arc? The two participants, referred to throughout as P1 and P2, were recruited as senior MEng Computing students at Imperial College London. Both performed six recorded refactoring sessions each on a new order-processing fixture (`user-study-fixture/`) structurally disjoint from the library fixture used in §5.2.2. Both also produced an independent labelling of the 45-session set *before* participating; that work feeds the inter-rater agreement reported in §5.2.1 and is not repeated here.

Descriptive findings on the participants' sessions. P1 and P2 each performed six recorded refactoring sessions on the order-processing fixture: 12 sessions in total. The playbook gives each session a what-not-how prompt naming the area of code to address but never the IntelliJ action to use, the testing or commit cadence to follow, or the divergence kinds the detector surfaces.[1] Because the playbook deliberately omits pre-declared `expected_kinds`, the participants' sessions admits no precision/recall claim of the form reported in §5.2.2; what it does admit is a descriptive per-kind distribution and a descriptive per-session count, which Table 5.12 gives in full.

The aggregate per-kind distribution across the 43 divergence points is IDE-replay 46.5%, hygiene 37.2%, rework 11.6%, ordering 4.7%. Both participants registered 0 divergence points on session S04, the cross-class validation consolidation prompt. The playbook for that session asks

[1] Quotes throughout this section are paraphrased from session-end notes taken during the post-study interview; full audio transcription was not feasible within the study budget. The source notes are retained as private study artefacts and are not reproduced here.

session	IDE_REPLAY	REWORK	HYGIENE	ORDERING	total
P1 S1	5	0	1	1	7
P1 S2	0	0	3	0	3
P1 S3	5	2	1	0	8
P1 S4	0	0	0	0	0
P1 S5	1	0	1	0	2
P1 S6	0	0	0	0	0
P2 S1	3	0	2	0	5
P2 S2	2	0	4	0	6
P2 S3	1	0	2	1	4
P2 S4	0	0	0	0	0
P2 S5	2	0	1	0	3
P2 S6	1	3	1	0	5
total	**20**	**5**	**16**	**2**	**43**

Table 5.12: Per-session divergence-point counts across the 12-session participants' sessions. Both participants registered 0 *divergence points on session S04, a cross-class duplication consolidation task that both executed as straight manual edits; the remaining sessions exercise all four kinds in proportions broadly consistent with the 45-session set.*

the participant to consolidate three near-identical validation blocks into a single home — a delete-an-inline-block plus delete-a-dead-method task. Both participants executed it as straight manual edits with no rework, no skipped tests, no manual-where-IDE-could-replay candidate, and no reorder window, so the detector legitimately had nothing to surface. Used cautiously, this is the closest the chapter comes to an external true-negative observation: when the task can be discharged in a couple of clean edits and the participants do so, the detector stays quiet. Both participants identified IDE-replay as the most actionable kind in interview, with one participant noting that they "didn't realise that IntelliJ could do all of these refactorings" so didn't use them; both identified ordering as the least actionable, with one asking "how can I actually apply that in the future without knowing".

Behavioural trajectory across the six-session arc. A second question from the participants' sessions is whether per-kind divergence-point rates evolve as participants accumulate feedback. Table 5.13 aggregates per-kind divergence-point counts across the two participants, session by session. Two kinds show trajectory signal. The IDE-replay aggregate falls from 8 in S1 to 1 in S6, an 88% reduction with a mid-arc spike in S3; per-participant, P1 falls from 5 to 0 and P2 falls from 3 to 1, both trending down across the arc. The hygiene aggregate peaks at 7 in S2 — consistent with both participants first seeing commit-cadence and test-cadence feedback at the end of S1 and applying it from S3 onwards — and then falls to 1 in S6. Both participants self-reported the corresponding behavioural change in interview: one said they "ran tests after every change and committed more and used the provided automated tools", and the other said they "used the IDE refactoring methods more frequently". The IDE-replay and hygiene patterns align with what the participants said they did, but with only $n = 2$ participants over six sessions, this is a descriptive observation rather than evidence of a causal effect.

kind	S1	S2	S3	S4	S5	S6	Δ
IDE_REPLAY	8	2	6	0	3	1	−7
HYGIENE	3	7	3	0	2	1	−2
ORDERING	1	0	1	0	0	0	−1
REWORK	0	0	2	0	0	3	+3

Table 5.13: Per-kind divergence-point trajectory across the six-session arc, summed across both participants. IDE-replay and hygiene show a downward trajectory consistent with self-reported behavioural change; ordering and rework are episodic, firing on the sessions whose prompts inherently involve them.

The remaining two kinds do not show trajectory signal. Ordering fires rarely on the participants' sessions — 2 positive-magnitude divergence points in total across the two participants and six sessions each, where the synthesiser found a permutation that strictly beats the user's order — consistent with the headline finding of §5.1.3 that the score formula is insensitive to commuting reorders and that positive-magnitude reorder alternatives are concentrated on the small fraction

of windows containing genuinely non-commuting pairs. Rework is episodic, spiking on the participants' hardest sessions where contiguous undo/redo cycles drive several flags from a single sequence of mis-clicks. One participant raised this as a specific metric-design complaint in interview, observing that rework "picked up on undos + redos a lot (in a bad way) [—] over penalised" on contiguous-step sequences caused by mistyped keys. The Table 5.13 pattern is consistent with that account: the 3 rework flags concentrated in S6 cluster within the single most complex session of the arc, where P2 produced all of those flags from a contiguous sequence of small edits.

The S3 and S6 rework spikes are a reminder that the six sessions are not interchangeable: S2 (magic-number renames) and S4 (cross-class consolidation) are short, mechanical sessions where divergence-point counts collapse for both participants regardless of where they sit in the arc, while S3 and S6 are structurally harder sessions where the opportunity for rework is built into the playbook prompt. Cross-session comparisons of raw divergence-point counts or of the within-session process score are therefore confounded by task difficulty, and the per-kind columns of Table 5.13 are the cleaner trajectory signal: IDE-replay and hygiene fall across the arc on the kinds where behavioural change was possible, while rework rises on the hard sessions for both participants. The same confound applies similarly to the agent-comparison reading below.

Process scores under the production and gain-stripped weightings. The within-session process score itself can be partially separated from task difficulty by inspecting it twice: once under the production weights, and once under a weighting that sets the cleanliness-gain weight W_g to zero. The cleanliness-gain term is the only term in the score formula that scales with how much the code itself improved over the session, and the ablation in §5.1.2 already showed that this term acts as a regularizer on divergence-point ranking rather than reshaping it. W_g is the largest weight in the production formula by design, since the anti-gaming axiom of §3.2 requires the cleanliness reward to outweigh any process-side shortcut a developer could plausibly take by trading off code quality for procedural discipline; zeroing W_g in a free-form deployment would invite exactly that gaming. The participants' sessions are not free-form: the playbook fixes the structural change each session must produce, so the cleanliness outcome is set by whether the participant completed the prompt rather than by anything the participant could trade off, and the gain-stripped view is a legitimate read on the process-discipline terms alone for this experiment. Zeroing W_g leaves the broken-time, skipped-tests, manual-when-IDE, length-bonus, and commit-gap terms in place, so the remaining score asks "did the user execute the process well" rather than "did the code end up cleaner". Table 5.14 reports both views across the six-session arc for P1, P2, and the agent.

actor	S1	S2	S3	S4	S5	S6	mean
P1 (production)	25	78	18	97	56	57	55.2
P1 (gain = 0)	25	28	35	47	40	50	37.5
P2 (production)	16	68	35	97	31	31	46.3
P2 (gain = 0)	16	18	31	47	31	40	30.5
Agent (production)	0	79	30	100	72	39	53.3
Agent (gain = 0)	37	40	40	50	42	35	40.7

Table 5.14: Trajectory-final process score across the six-session arc for each actor under the production weighting and under a copy of the weighting with the cleanliness-gain weight W_g set to zero. The production view reads the actor's outcome and process together; the gain-stripped view reads the process alone.

Two patterns sit in the table. First, the shape of the production trajectory is structurally similar across all three actors: S2 and S4 are the high points, S1 and S3 are the low points, S5 and S6 land in between. This is the task-difficulty signature. S2 (magic-number extraction) and S4 (cross-class duplication consolidation) reward the actor with a large positive cleanliness delta for a small number of edits; S1 (renames) and S3 (within-class duplication) demand careful intermediate-state management and offer less cleanliness gain to compensate. The agent and the participants land on the same shape because the shape is set by the playbook prompts, not by how well the actor executed them.

Second, the gain-stripped trajectory shows a pattern that the production trajectory hides. P1's process score under $W_g = 0$ rises nearly monotonically across the arc, from 25 at S1 to 50 at S6, with one -7 dip at S5. S6 sits at the baseline ceiling of 50, meaning P1 incurred essentially no process-discipline penalty on the structurally hardest session of the arc. P2 trends the same direction less consistently: $16 \to 18 \to 31 \to 47 \to 31 \to 40$. The agent's gain-stripped scores

show no arc-position trend at all: $37, 40, 40, 50, 42, 35$, varying with task structure rather than session number. The playbook orders the six sessions in roughly increasing structural complexity (S1 renames through to S6 multi-step reshape), so the participants' improvement is happening despite progressively harder tasks, not riding on easier ones. The agent shows no such trend, suggesting that the qualitative between-session course-corrections discussed in §5.3.2, Thread 1 do not compound into a monotonic improvement on the process-discipline subscore over the arc.

The direction of the participants' improvement is what the tool's feedback is designed to produce, but the design has no control condition (no participant performed the arc without dashboard feedback between sessions) and $n = 2$ participants is too small to support a causal claim. The gain-stripped subscore is largely insulated from how familiar the participant is with the codebase itself — the terms active under $W_g = 0$ are commit cadence, skipped tests, and manual-when-IDE rate, none of which depend on knowing the code better. What the design does not separate is the participants intuiting what the dashboard rewards and adjusting behaviour to satisfy it (a score-gaming reading), or becoming more comfortable with the playbook idiom across the arc (a task-style-familiarity reading). The threats-to-validity chapter (§??) discusses these. What the table does support is a narrower claim: the production score's session-level value is dominated by the cleanliness-gain term, so the production view alone does not show what changes in the participants' process across the arc, and the gain-stripped view does.

One additional qualitative finding does not show in the trajectory table but matters for future tuning: hygiene also produces a false-positive shape, firing when a participant is thinking rather than stuck. One participant flagged this in the interview as was "penalised for not running tests when wasn't quite finished but just was thinking", and the pattern is reflected in the S2 peak for hygiene more generally. Both this finding and the rework over-penalisation are reported here as candidate metric-tuning directions rather than detector bugs: the divergence points are accurate readings of what the trajectory contains, but the metric design treats some legitimate-pause and mistyped-key cases more harshly than the participant would.

Per-step inspection as a usability win. Both participants singled out the dashboard's per-checkpoint detail panel as a positive part of the interface, independently of the four detector kinds. One participant said the per-checkpoint view made it easy to "see what action you made to cause the issues to happen", and that the trajectory graph showing the highest score reachable from each point onwards was "egging you on to be a better person" (i.e. the alternative-trajectory overlays were read as constructive rather than punitive). The other participant said they could "see the specific point you went wrong", valued the cleanliness-signal "breakdowns" on each step, and called the dashboard "easy to track session at specific points and code smells". These observations validate the per-step smell-attribution and metric-shift breakdown described in §4.4 as a usability feature the participants engaged with, not just an architectural by-product of how the report is assembled.

5.3.2 Agent comparison

An automated coding agent — Claude Code (Anthropic, model `claude-opus-4-7`, run on 2026-05-21) — performed the same six-session playbook on the same fixture. Between sessions, the human facilitator pasted the markdown output of the `feedbackForAgent` task (`tool/analysis/.../experiment/FeedbackFo` into the agent's prompt at the start of the next session. The task renders an analysis report's divergence points and trajectory advice into the same plain-text summary that a human participant would read on the dashboard, so the agent received the same information the human participants did, just in textual rather than rendered form. The agent honoured a small setup contract before any session ran: tests were invoked via a wrapper script that brackets the gradle call with `TEST_RUN_STARTED` / `TEST_RUN_FINISHED` events in the plugin trace, and commits were made via standard `git commit` which the plugin's reflog watcher captured as `GIT_COMMIT` events. Apart from starting and stopping the plugin's recording at session boundaries and pasting the previous session's feedback into the next prompt, no human intervened during a session.

Table 5.15 reports the agent's per-kind divergence-point totals against each participant's six-session total from Table 5.13. The agent produced 5 divergence points across six sessions, against 20 from P1 and 23 from P2 over the same six sessions each — but the *shape* of the comparison is the headline, not the count. All five of the agent's divergence points are IDE-replay; rework, hygiene, and ordering fired zero times across the entire six-session arc. Both participants exercise all four kinds.

kind	Agent	P1	P2	Agent / P1	Agent / P2
IDE_REPLAY	5	11	9	0.45	0.56
REWORK	0	2	3	0.00	0.00
HYGIENE	0	6	10	0.00	0.00
ORDERING	0	1	1	0.00	0.00
total	**5**	**20**	**23**	**0.25**	**0.22**

Table 5.15: Per-kind divergence-point totals for the agent over six sessions on the user-study fixture, against each participant's six-session total separately so the comparison is run on the same session count for each row.

Table 5.16 mirrors the human trajectory table for direct visual comparison. The agent's divergence-point counts session by session are $0, 0, 1, 0, 2, 2$, and the final-checkpoint process scores are $0, 79, 30, 100, 72, 39$. Neither row shows monotonic improvement, for the same task-difficulty reason noted earlier in this section: S2 and S4 are short, mechanical sessions where everyone — both participants and the agent alike — produces near-zero divergence points and runs the score close to the ceiling; S3 and S6 are structurally harder sessions where the human cohort's rework count spikes as well. The cleaner trajectory signal is the per-kind columns rather than the per-session score endpoints.

kind	S1	S2	S3	S4	S5	S6	Δ
IDE_REPLAY	0	0	1	0	2	2	+2
REWORK	0	0	0	0	0	0	0
HYGIENE	0	0	0	0	0	0	0
ORDERING	0	0	0	0	0	0	0
final score	0	79	30	100	72	39	—

Table 5.16: Agent per-session divergence-point trajectory and final-checkpoint process scores, mirroring Table 5.13 for the human cohort.

Reading the per-kind columns left-to-right tells a different story from the score row. Rework, hygiene, and ordering held at zero across all six sessions, consistent with the procedural-discipline contract written into the agent's setup prompt being honoured throughout: tests via the wrapper after every change, commits at the boundaries the playbook prescribes, no undo cycles. The S1 dashboard did surface an admitted-but-not-improving ordering window (a permutation existed that reached the same end state but did not strictly beat the agent's; under the strict magnitude > 0 filter of §3.5.1 this is informational rather than a divergence point), and the agent acknowledged it in writing after the session; the kind never re-surfaced, but no controlled-comparison claim is made on a window whose magnitude is non-positive.

Behavioural threads from the agent's own reflections. The aggregate trajectory table flattens what is, on inspection, a substantively richer story. The full transcript reproduced in Appendix A shows the agent reading each session's feedback, diagnosing its own anti-patterns in writing, and adjusting approach between sessions in ways the per-kind counts only partially capture. Three threads stand out.

Thread 1: commit cadence, over-corrected and then resolved. S1 produced the worst-scoring session of the arc: one bulk commit at the end of all three renames, a session-long stretch without committing, a 28% build-broken rate (the build was broken on 2 of 7 checkpoints), one test regression, and a process score that fell from 50 to 0 — the largest single-session drop on the arc. The post-session feedback fired three advice items at critical and warning levels. The agent's response, written before S2 began, was a direct procedural correction: "commit smaller and more often — one commit per rename (or per logical step), not one at the end. Test after each step." S2 implemented exactly this. The agent did three steps as three separate `git commit` calls, each preceded by `./agent-test`; the resulting session produced 0 divergence points, 0 advice items, and the process score climbed from 50 to 79 — a +79-point swing across one session boundary attributable to a single concrete procedural change.

The agent then over-corrected in the opposite direction. Its pre-S3 statement was "I'll aim to commit even more granularly (each individual constant/edit, not each step)", and S3 fired three

IDE-replay divergence points (one at magnitude −6), `REFACTORING_INTRODUCED_SMELLS`, and a process score drop from 50 to 30. By S6 the agent had revised the rule a second time, but in a more interesting direction: the S6 advice flagged an 8-checkpoint commit stretch and a 37% build-broken rate (3 of 8 checkpoints), and the agent's diagnosis identified the cause as a structural tension the rule itself created: "for an Extract Method, do it as a single atomic edit (insert new method first, then replace call site) so the build never breaks mid-step." The agent had discovered, by being penalised for it twice, that "commit more often" and "don't break the build" are in conflict when an edit is structurally multi-step, and resolved the conflict by switching to larger atomic diffs rather than finer-grained commits. This is the most substantive behavioural-trajectory finding in the arc: the agent did not just reduce its DP count, it revised its operational model of the trade-off the advice was implicitly pointing at. Placing the broken-build percentages alongside the human cohort's puts the size of the issue in context: across the arc the agent's per-session broken-build percentage (broken wall-clock time over total session wall-clock time) averages 6.5% with a peak of 24% on S6, while P1 averages 7.3% with a peak of 23% on S1 and P2 averages 14.0% with a peak of 52% on S1. The agent is comparable to P1 on average but P2's worst session is more than twice the agent's worst. These per-session percentages are what surface as the `BUILD_OFTEN_BROKEN` dashboard advice when they cross the warning or critical thresholds; the agent's wrapper script forces a test run at every committed checkpoint but does not prevent intermediate broken states between checkpoints, which is exactly the failure mode the atomic-edit rule above addresses.

Thread 2: smell-introduction feedback produced a specific self-critique. After S3 the agent fired `REFACTORING_INTRODUCED_SMELLS` alongside the process-score drop, and the agent's written response identified its own anti-pattern by name: "methods like `chargeCash`/`refundVoucher` are arguably trivial wrappers that didn't earn their keep," and "could have made `formatAddress` cleaner." The follow-on sessions support the diagnosis: S4 produced zero divergence points and zero advice items, S5 produced two IDE-replay flags but no `REFACTORING_INTRODUCED_SMELLS`, and the smell-introduction advice did not fire again until S6 — where the agent attributed it to multi-edit extract-method mechanics rather than the trivial-wrapper anti-pattern from S3. The smell-introduction feedback drove a course correction that holds for two consecutive clean sessions before resurfacing under a different cause.

Thread 3: agent acknowledges the structural IDE-replay limitation. The persistent failure mode is IDE-replay. All five of the agent's divergence points are IDE-replay flags, and the count does not decline across the arc; it is concentrated on the structurally heavier sessions (S3, S5, S6) where the agent performed enough structural refactoring to surface any divergence points at all. The agent has no IDE refactor surface to switch to: Claude Code edits files via text tool calls, not through IntelliJ's Refactor menu, so every refactoring it performs is classified as a manual edit by the detector. The agent itself acknowledges the structural nature of this in its S6 reflection, observing that "the IDE-replay flags persist — manual extracts consistently underperform the IntelliJ refactoring; in a real run I'd use the Extract Method action." The signal is real and the recommendation correct, but the agent lacks the tool to execute it. The agent cannot drive the IDE-replay count toward zero however well it works, and this is the cleanest example in the arc of feedback being correctly interpreted but structurally inactionable. A setup that exposed IDE refactor actions as agent tool calls would close the gap - that work is named in the future-work section.

The two failure profiles are nearly complementary. Humans are *procedurally noisy and architecturally capable*: high IDE-replay and hygiene counts, scattered rework on the hard sessions, but able to switch to IntelliJ refactor actions when they remember to. The agent is *procedurally disciplined and architecturally limited*: zero on rework, hygiene, and ordering — the three kinds where the tool measures process cadence and sequencing — and IDE-replay only on the sessions where it performed structural refactoring at all, because the actuator that would resolve IDE-replay does not exist in its interface. The tool surfaces both kinds of failure correctly; neither profile is the "right" one to target, and the comparison demonstrates that the detector's four kinds are pulling apart genuinely different dimensions of refactoring practice rather than collapsing onto a single dimension.

The agent comparison is descriptive at $n = 1$ agent over six sessions with one model version on one date; it is illustrative of the detector's behaviour on a non-human actor rather than a generalisable claim about coding agents. A multi-agent extension, and an agent-side IDE refactor

capability that would let the IDE-replay count actually respond to feedback, are flagged as future work.

Chapter 6

Project Plan and Timetable (Jan–Jun 2026)

Work completed so far (as of 21 Jan 2026).

- Extensive related-work research completed (refactoring definitions, endpoint evaluation, smells/metrics, mining, sequence generation, IDE logging), with an initial BibTEX file in progress.
- Early drafts written for the Introduction and Background/Related Work chapters

Timeline in 5 phases (Jan → end of June).

- **Late Jan–Feb:** define refactoring *episodes* and system outputs (trace format, step definition, required explanations), pick an initial metric (*Metric v0*), and finalise the IDE plugin plan.
- **Feb–Mar:** implement the IDE plugin and begin collecting a first batch of refactoring episodes/traces using the plugin.
- **Mar–Apr:** build the offline analysis pipeline (episode → feature extraction → scoring) and implement baseline explanation outputs (per-step breakdown and "what got worse/why" style diagnostics).
- **Apr–May:** implement reference-trajectory generation (v1) and divergence detection (developer trajectory vs. reference trajectory), producing explanations of where/why the paths differ.
- **May–Jun:** run evaluation (human traces + simulated "unclean" variants, with optional open-source mining if feasible), consolidate results, and complete the final report write-up and polishing.

Chapter 7

Ethical Issues and Data Handling Plan

The main ethical considerations in this project come from collecting developer traces via an IDE plugin, and from the possibility that the tool could be misused as a form of monitoring. The technical work itself is not safety-critical, but the data involved can still be sensitive because it relates to how people work and may include source code artefacts.

If I collect refactoring episodes from human participants, I will use informed consent and make it clear what is logged (e.g., refactoring commands, checkpoints/snapshots, basic metadata like timestamps, and build/test outcomes) and what is *not* logged (e.g., personal accounts, unrelated browsing, or continuous keystroke-level recordings unless genuinely required). I will follow a data minimisation approach: only collect the smallest amount of information needed to reconstruct refactoring steps and compute the evaluation metrics. Participants will be able to withdraw, and I will delete their data if requested.

Source code can be commercially sensitive, even when the research goal is about refactoring rather than the code itself. To avoid IP issues, I plan to run participant studies on toy projects and/or open-source repositories, rather than proprietary code. If open-source repositories are used, I will respect licensing requirements and avoid re-publishing large code excerpts in the dissertation. If I later want to evaluate on any non-public codebase, I would only do so with explicit written permission and with clear constraints on what is stored and shared.

Even without names, traces can sometimes be identifying (e.g., file paths, commit messages, timestamps, distinctive code structure). For any stored traces, I will remove or hash direct identifiers (names, usernames, machine paths, repository URLs) and avoid publishing raw logs. In the dissertation, I will present results in aggregated form where possible (e.g., across episodes), and I will be careful not to "name and shame" individual developers or projects when discussing low-quality trajectories.

A tool that evaluates a developer's "process quality" could be repurposed for workplace surveillance or performance management. Although that is not the intended use, it is important to acknowledge the risk. I will frame the tool as a developer support and coaching system (actionable feedback about code/process), not as a ranking system for people. In particular, I will avoid outputs that resemble employee scorecards or leaderboards, and I will focus on episode-level technical explanations rather than person-level judgements.

All collected data (traces and any derived metrics) will be stored securely with access limited to the project work. I will keep raw logs only for as long as needed to run experiments and compute results, and then retain only the derived, anonymised summaries required for analysis and write-up. If any data needs to be shared with a supervisor or included in an appendix, it will be anonymised and minimised to reduce the chance of leaking sensitive information.

Appendix A

Agent session transcript

The complete session-by-session transcript of the six-session agent run reported in §5.3.2 is included here verbatim. Each session shows the agent's tool calls, intermediate output, the markdown feedback rendered by `FeedbackForAgent` at session end, and the agent's reflective response before the next session began. The transcript is the source for every quoted passage in §5.3.2.

Bibliography

[1] William F. Opdyke. *Refactoring Object-Oriented Frameworks*. PhD thesis, University of Illinois at Urbana-Champaign, Urbana-Champaign, IL, USA, 1992.

[2] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.

[3] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 2 edition, 2018.

[4] Tom Mens and Tom Tourwé. A survey of software refactoring. *IEEE Transactions on Software Engineering*, 30(2):126–139, 2004.

[5] Emerson R. Murphy-Hill, Chris Parnin, and Andrew P. Black. How we refactor, and how we know it. In *Proceedings of the 31st International Conference on Software Engineering (ICSE)*, pages 287–297, 2009.

[6] Miryung Kim, Danny Cai, and Sunghun Kim. An empirical investigation into the role of API-level refactorings during software evolution. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, pages 151–160, 2011.

[7] Pouria Alikhanifard and Nikolaos Tsantalis. A novel refactoring and semantic aware abstract syntax tree differencing tool and a benchmark for evaluating the accuracy of diff tools. *ACM Transactions on Software Engineering and Methodology*, 34(2):Article 40, 2025.

[8] Jean-Rémy Falleri and Matias Martinez. Fine-grained, accurate, and scalable source code differencing. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM, 2024.

[9] Dhruv Gautam, Spandan Garg, Jinu Jang, Neel Sundaresan, and Roshanak Zilouchian Moghaddam. RefactorBench: Evaluating stateful reasoning in language agents through code. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[10] Islem Bouzenia and Michael Pradel. Understanding software engineering agents: A study of thought-action-result trajectories. 2025.

[11] Eman Abdullah AlOmar et al. Software refactoring research with large language models: A systematic literature review. *Information and Software Technology / Journal of Systems and Software*, 2025. Article S0164121225004315.

[12] Veselin Raychev, Max Schäfer, Manu Sridharan, and Martin Vechev. Refactoring with synthesis. In *Proceedings of the 2013 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 339–354. ACM, 2013.

[13] Yu Lin, Xin Peng, Yumin Cai, Danny Dig, Li Zhang, and Wenyun Zhao. Interactive and guided architectural refactoring with search-based recommendation. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 535–546. ACM, 2016.

[14] Mark Harman and Laurence Tratt. Pareto optimal search-based refactoring at the design level. *Journal of Systems and Software*, 80(4):522–534, 2007.

[15] M. Mohan and Des Greer. Using a many-objective approach to investigate automated refactoring. *Information and Software Technology*, 2019.

[16] Wang et al. 2018.

[17] Nikolaos Nikolaidis et al. A metrics-based approach for selecting among various refactoring candidates. *Empirical Software Engineering*, 2024.

[18] Matheus Paixão et al. An empirical study of cohesion and coupling: Balancing optimisation and disruption. *IEEE Transactions on Evolutionary Computation*, 2017.

[19] Naouel Moha et al. Decor: A method for the specification and detection of code and design smells. *IEEE Transactions on Software Engineering*, 2010.

[20] Thiago Paiva et al. On the evaluation of code smells and detection tools. *Journal of Software Engineering Research and Development*, 2017.

[21] Francesca Arcelli Fontana et al. Antipattern and code smell false positives: Preliminary conceptualization and classification. In *Proceedings of a workshop on code smells / antipatterns*, 2016.

[22] Simone Scalabrino et al. A comprehensive model for code readability. *Journal of Systems and Software*, 2018.

[23] Danilo Silva et al. Refdiff: Detecting refactorings in version histories. In *Proceedings of the International Conference on Mining Software Repositories (MSR)*, 2017.

[24] Reudismam Rolim, Gustavo Soares, Loris D'Antoni, Oleksandr Polozov, Sumit Gulwani, Sherry Ge, and Neha Rungta. Learning syntactic program transformations from examples. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, pages 404–415. IEEE, 2017.

[25] Vittorio Cortellessa, Daniele Di Pompeo, Vincenzo Stoico, and Michele Tucci. Many-objective optimization of non-functional attributes based on refactoring of software models. *Information and Software Technology*, 2023.

[26] Lerina Chen and Shinpei Hayashi. MORCoRA: Multi-objective refactoring recommendation considering review availability. *International Journal of Software Engineering and Knowledge Engineering*, 34(12):1919–1947, 2024.

[27] Vahid Alizadeh, Marouane Kessentini, Ali Ouni, Mohamed Wiem Mkaouer, Mel Ó Cinnéide, and Yuanfang Cai. An interactive and dynamic search-based approach to software refactoring recommendations. *IEEE Transactions on Software Engineering*, 46(9):932–961, 2020.

[28] Valentina Pantiuchina, Fiorella Zampetti, Simone Scalabrino, et al. Why developers refactor source code: A mining-based study. *ACM Transactions on Software Engineering and Methodology*, 29(4), 2020.

[29] Germán Robredo, Isabel Gonzales, Yania Crespo, et al. What were you thinking? an llm-driven large-scale study of refactoring motivations in open-source projects, 2025. arXiv:2509.07763.

[30] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yuandong Zhang. Training software engineering agents and verifiers with SWE-Gym. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025.

[31] Anna Maria Eilertsen. A study of refactorings during software change tasks. *Journal of Software: Evolution and Process*, 2024.

[32] Kostadin Damevski, Thomas Fritz, and David Shepherd. Mining sequences of developer interactions in visual studio for usage smells. *IEEE Transactions on Software Engineering*, 43(4):359–371, 2017.

[33] Wouter Poncin, Alexander Serebrenik, and Mark van den Brand. Process mining software development workflows. In *Proceedings of the 15th European Conference on Software Maintenance and Reengineering (CSMR)*, pages 379–382. IEEE, 2011.

[34] Manaal Basha, Aimêe M. Ribeiro, Jeena Javahar, Cleidson R. B. de Souza, and Gema Rodríguez-Pérez. CodeWatcher: IDE telemetry data extraction tool for understanding coding interactions with LLMs. In *Proceedings of the 41st IEEE International Conference on Software Maintenance and Evolution (ICSME), Tool Demonstration Track*, 2025.

[35] Said Foutsekh, Aiko Yamashita, Fabio Petrillo, Mourad Khomh, and Yann-Gaël Guéhéneuc. Developer interaction traces backed by ide screen recordings from think-aloud sessions. In *Proceedings of the 15th International Conference on Mining Software Repositories (MSR) (Data Showcase)*. ACM, 2018. Dataset paper providing IDE interaction traces with synchronized screen recordings.

[36] Diego Cedrim, Alessandro Garcia, Melina Mongiovi, Rohit Gheyi, Leonardo Sousa, Rafael de Mello, Baldoino Fonseca, Márcio Ribeiro, and Antonio Chávez. Understanding the impact of refactoring on smells: A longitudinal study of 23 software projects. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 465–475. ACM, 2017.

[37] Stas Negara, Nicholas Chen, Mohsen Vakilian, Ralph E. Johnson, and Danny Dig. A comparative study of manual and automated refactorings. In *Proceedings of the 27th European Conference on Object-Oriented Programming (ECOOP)*, volume 7920 of *LNCS*, pages 552–576. Springer, 2013.

[38] Mohsen Vakilian, Nicholas Chen, Stas Negara, Balaji Ambresh Rajkumar, Brian P. Bailey, and Ralph E. Johnson. Use, disuse, and misuse of automated refactorings. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, pages 233–243. IEEE, 2012.

[39] Yida Tao and Sunghun Kim. Partitioning composite code changes to facilitate code review. In *Proceedings of the 12th IEEE/ACM Working Conference on Mining Software Repositories (MSR)*, pages 180–190. IEEE, 2015.

[40] Marco Di Biase, Magiel Bruntink, Arie van Deursen, and Alberto Bacchelli. The effects of change decomposition on code review: A controlled experiment. *PeerJ Computer Science*, 5:e193, 2019.

[41] Kim Herzig and Andreas Zeller. The impact of tangled code changes. In *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR)*, pages 121–130. IEEE, 2013.

[42] Gunnar Kudrjavets, Nachiappan Nagappan, and Ayushi Rastogi. Do small code changes merge faster? A multi-language empirical investigation. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*. IEEE, 2022.

[43] G. Ann Campbell. Cognitive complexity: A new way of measuring understandability, 2018. SonarSource white paper, not peer-reviewed.

[44] Marvin Muñoz Barón, Marvin Wyrich, and Stefan Wagner. An empirical validation of cognitive complexity as a measure of source code understandability. In *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. ACM, 2020. Best Full Paper award.

[45] Luigi Lavazza, Abdallah Z. Abualkishik, Geng Liu, and Sandro Morasca. An empirical evaluation of the "Cognitive Complexity" measure as a predictor of code understandability. *Journal of Systems and Software*, 197:111561, 2023.

[46] Shyam R. Chidamber and Chris F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994.

[47] Ilja Heitlager, Tobias Kuipers, and Joost Visser. A practical model for measuring maintainability. In *Proceedings of the 6th International Conference on the Quality of Information and Communications Technology (QUATIC)*, pages 30–39. IEEE, 2007.

[48] Raymond P. L. Buse and Westley R. Weimer. Learning a metric for code readability. *IEEE Transactions on Software Engineering*, 36(4):546–558, 2010.

[49] Radu Marinescu. Detection strategies: Metrics-based rules for detecting design flaws. In *Proceedings of the 20th IEEE International Conference on Software Maintenance (ICSM)*, pages 350–359. IEEE, 2004.

[50] James M. Bieman and Byung-Kyoo Kang. Cohesion and reuse in an object-oriented system. *ACM SIGSOFT Software Engineering Notes*, 1995. Introduces Tight Class Cohesion (TCC) metric.

[51] Stefan Wagner, Andreas Goeb, Lars Heinemann, Michael Kläs, Constanza Lampasona, Klaus Lochmann, Patrick Mäder, Reinhold Plösch, Matthias Saft, Jürgen Streit, and Adam Trendowicz. Operationalised product quality models and assessment: The Quamoco approach. *Information and Software Technology*, 2015.

[52] Jagdish Bansiya and Carl G. Davis. A hierarchical model for object-oriented design quality assessment. *IEEE Transactions on Software Engineering*, 28(1):4–17, 2002.

[53] Don Coleman, Dan Ash, Bruce Lowther, and Paul Oman. Using metrics to evaluate software system maintainability. *Computer*, 27(8):44–49, 1994.

[54] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.

[55] J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977.

[56] Eclipse Foundation. JDT/FAQ. Eclipse Wiki, 2026. Consulted 2026-05-18.

[57] Eclipse Foundation. Eclipse JDT Language Server (eclipse.jdt.ls). Eclipse Foundation project; reference implementation on GitHub, 2026. Newsletter article by Gorkem Ercan et al., Eclipse Newsletter, May 2017; reference implementation `eclipse-jdtls/eclipse.jdt.ls` on GitHub. Consulted 2026-05-18.

[58] Maurice G. Kendall. *Rank Correlation Methods*. Charles Griffin & Co., London, 1948.

[59] Alan Agresti. *Analysis of Ordinal Categorical Data*. John Wiley & Sons, Hoboken, NJ, 2 edition, 2010.

[60] William Webber, Alistair Moffat, and Justin Zobel. A similarity measure for indefinite rankings. *ACM Transactions on Information Systems*, 28(4):20:1–20:38, 2010.

[61] Michaela Saisana, Andrea Saltelli, and Stefano Tarantola. Uncertainty and sensitivity analysis techniques as tools for the quality assessment of composite indicators. *Journal of the Royal Statistical Society: Series A (Statistics in Society)*, 168(2):307–323, 2005.

[62] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 4765–4774, 2017.

[63] Erik Štrumbelj and Igor Kononenko. Explaining prediction models and individual predictions with feature contributions. *Knowledge and Information Systems*, 41(3):647–665, 2014.

[64] Tom Mens, Gabriele Taentzer, and Olga Runge. Analysing refactoring dependencies using graph transformation. *Software and Systems Modeling (SoSyM)*, 6(3):269–285, 2007.

[65] Hui Liu, Lu Yang, Zhendong Niu, Zhiyi Ma, and Weizhong Shao. Facilitating software refactoring with appropriate resolution order of bad smells. In *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 265–268. ACM, 2009.

[66] Younes Khrishe and Mohammad Alshayeb. An empirical study on the effect of the order of applying software refactoring. In *Proceedings of the 7th International Conference on Computer Science and Information Technology (CSIT)*, Amman, Jordan, 2016. IEEE.